

用matplotlib实现数据可视化



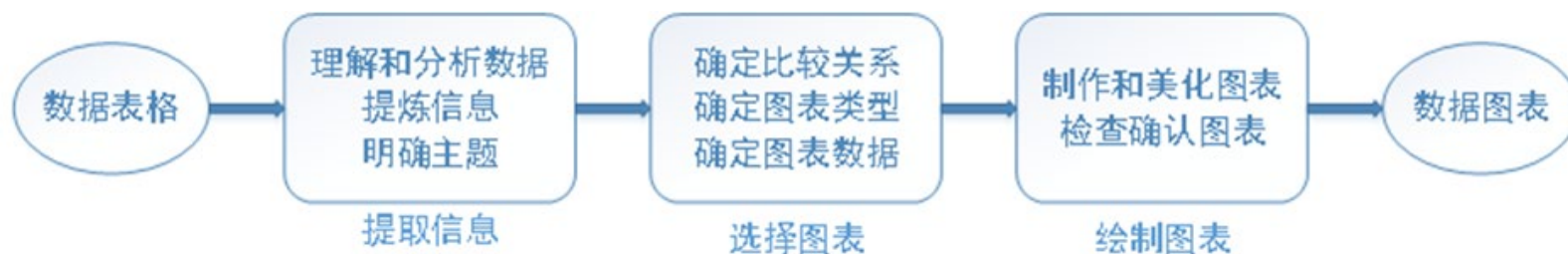
- ◆ 了解数据可视化概念
- ◆ 掌握图表中相关的术语
- ◆ 了解绘制图表的相关函数
- ◆ 掌握绘制散点图
- ◆ 掌握绘制折线图
- ◆ 掌握绘制条形图
- ◆ 掌握绘制直方图
- ◆ 掌握绘制面积图
- ◆ 掌握绘制饼图
- ◆ 掌握绘制子图
- ◆ 掌握图中的中文显示
- ◆ 掌握组合图形与标注
- ◆ 掌握保存图片

- 数据可视化是一个新术语，它不仅通过图表展示数据，更使用直观的方式将数据背后的信息展现出来。相对于大量的原始数据，人类可以快速理解数据可视化后的图表表达的信息。
- 目前成熟的图表已超过30种，每种图表有特定的实用场景和特点。数据有多种展示的方式，不同的展示方式突出的重点不同。下面是常用的几种图表：

图表类型	使用场景	例子
折线图	展示数据随时间或有序类别的波动情况的趋势变化	●某学生最近30天每日步行数 ●股票每日收盘价
直方图	分布分析展示数值在区间范围内的分布，可反映分类项目之间的比较，也可以用来反映时间趋势	●比较同一个班的学生在家庭作业、期中考试、期末考试和全部课程成绩的得分分布
散点图	研究不同变量间的关系	●男性与女性人群中不同年龄阶段得皮肤病的可能性 ●IQ测试得分和学习成绩之间的相关性
饼图	在一个空间或图上展示比例，展示不同部分所占百分比	●调查中的相应分类 ●一种特定技术的前五家公司的市场份额

图表类型	使用场景	例子
气泡图	用于展示三个变量之间的关系，横纵中表示其中两个变量，第三个变量用气泡的大小表示	●对比多种磁盘的性能，价格和销量。 x 轴表示价格， y 轴表示性能，气泡大小表示销量
核密度估计图	估计概率密度函数（非参数检验方法）	●股票、金融等风险预测
树状图	与组织结构有关的分析，即有明确的层次关系的数据	●展示企业组织结构
小提琴图	展示数值在区间范围内的分布	●不同性别对应的消费额

- 选择正确的图表和绘制技巧需要大量的练习和对比。有效的数据可视化有助于理解数据或传递信息。数据可视化的过程也需要对数据进行分析和提取，通过对数据的分析展现出其统计信息或隐藏的信息。数据可视化是数据分析后的一个过程，但是通常数据分析和数据可视化过程常常是交互进行的。
- 数据可视化的主要原则是根据目标读者的需求，结合通过分析数据能展示的问题，采用能精确呈现数据的特征的图表进行可视化。



- 从数据制作图表的过程可以知道，首先，需要对数据进行分析，明确需要表达的信息和主题。
- 根据要展示的信息选择图表类型，选择全部或部分数据展示在图表中。
- 最后将制作图表并对图表进行美化，确认图表表达了预期的主题。

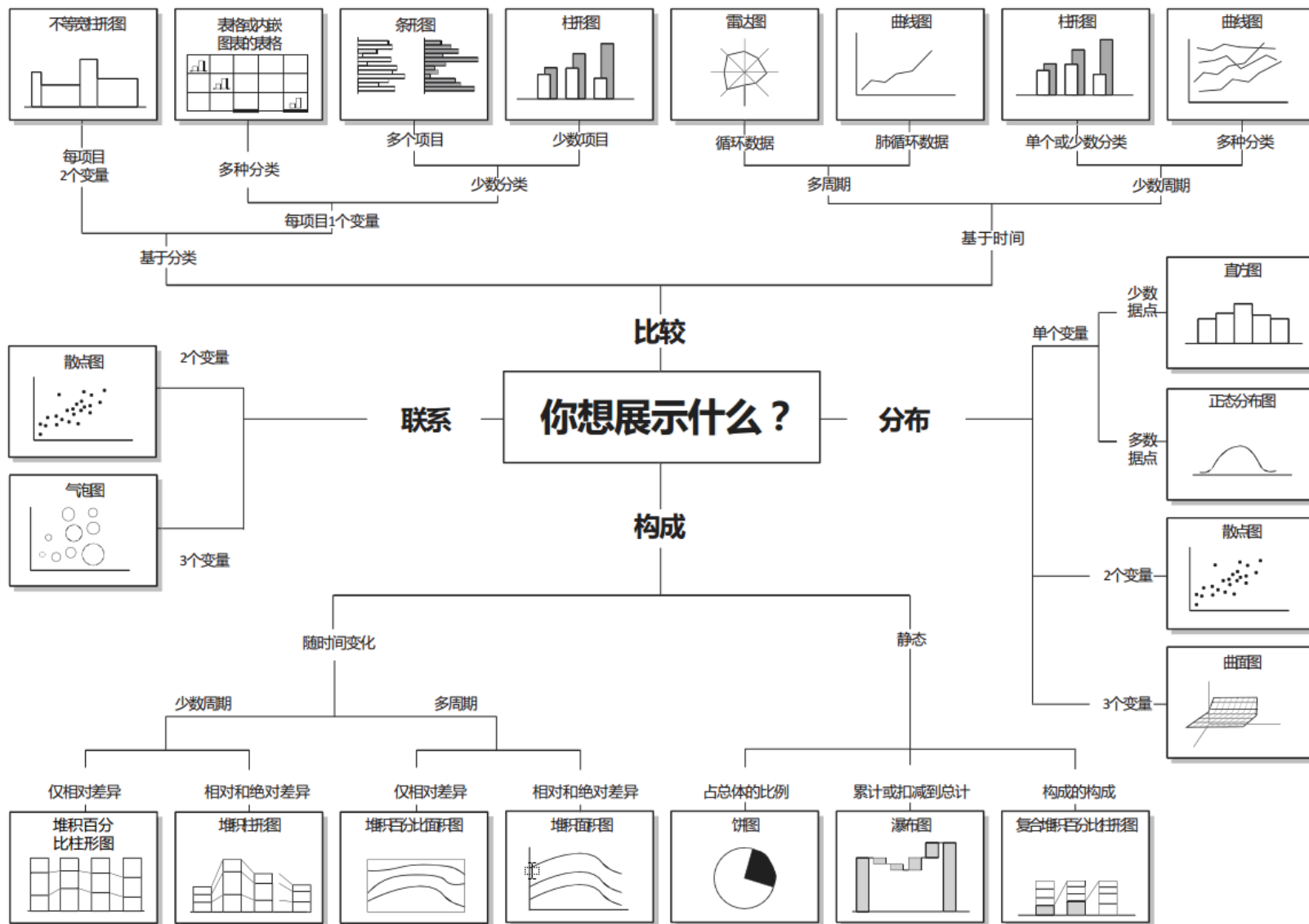
- 通过调整坐标轴或标注会给人误导，这种由统计图表调整造成的误导具有一定的“隐蔽性”。例如通过调整y轴的起始数值会给人一种直观上的错觉。这也是一些论文中突出优化效果的一种花招。
- 数据本身并不能确定图表类型，更重要的是通过数据表达的观点。选择图表类型有一定的技巧和原则，但是对于特定的问题需要独立分析。
- 美国专家Andrew Abela整理了一张图表类型选择指南。他根据需要表达的主题将图分为比较、分布、构成、联系四类。在每种类别下，根据不同情况选择不同的图表进行展示。



数据可视化基础



软件学院
School of Software

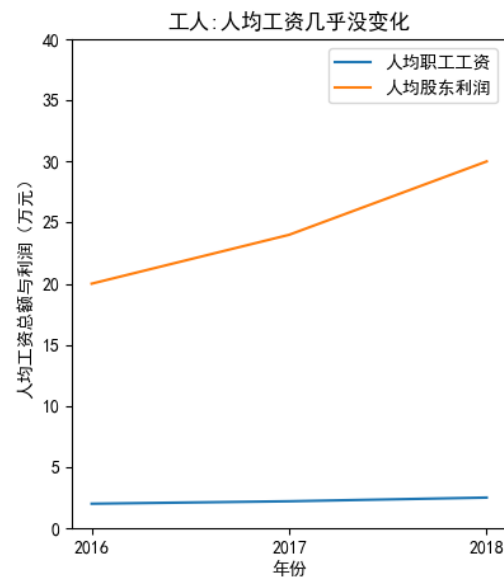
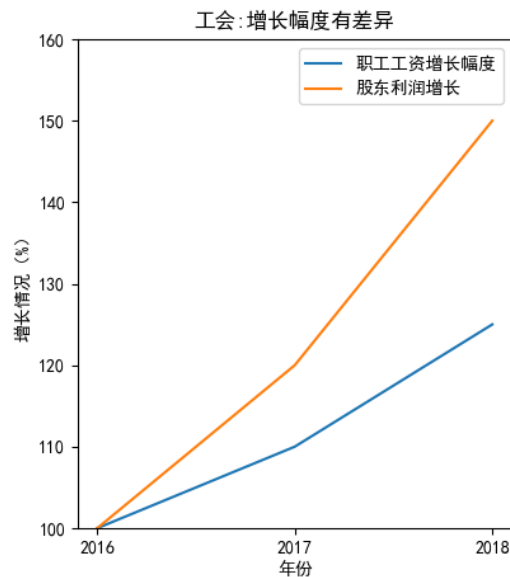
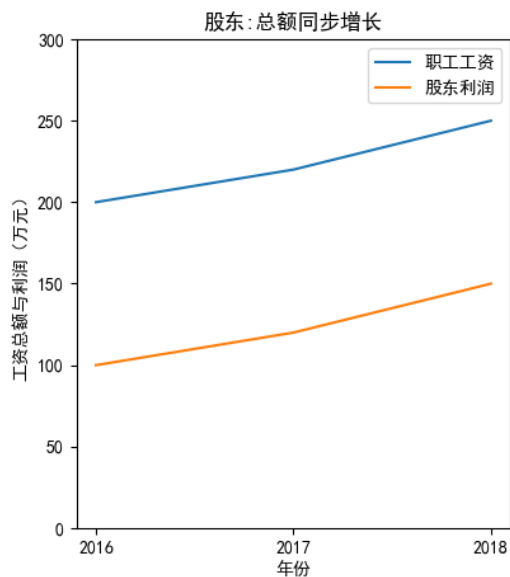


➤ 同样的数据，站在不同的立场上去解析，不同人发现的信息和观点可能不一样。不仅选用的图表类型不一样，即使采用同样的图表类型，通过不同的数据展示方式也可以强调不同观点。最重要的是，这种方法具有很强的迷惑性。

➤ 例:某公司有5个股东, 100个职工, 公司的股东利润和员工工资总额如表所示:

年份	2016年	➤2017年	2018年
职工工资 (万元)	200	➤220	250
股东利润 (万元)	100	➤120	150

➤ 公司股东, 工会和职工代表用同样的数据绘制了三张不同的图表。



➤ 从上面例子可以看出，从同样数据画出的三份图表，得出了三种不同的结论。股东、工会、职工三者是根据同一组数据从各自不同的观点出发，绘制出不同的统计图，进而根据它们得出了要表达的不同含义。应该说，三者都是有理有据的。同一组数据，不同的人站在不同的角度，可以画出不同的统计图，进而可以得出不同的结论。

➤ 图表展示的观点差异主要取决于几个方面：

- 1)要表达的观点和突出的重点；
- 2)从数据中提取的信息；
- 3)数据间的关系及提取信息的方法。

➤ 数据本身对图表的选择和设计也有一定的影响。

如采用直方图表示数据对比，当数据项差异悬殊时可能某些数据项数值过小无法得到量化的结果。如数据分类不当，饼图也无法传递有效的信息。这些情况都要对图表进行进一步的处理和美化。

● 使用Matplotlib绘图

- Python语言的matplotlib扩展包是一个提供了跨平台的2D图形库，它可以将数据用多种形式展示出来。Matplotlib可画出折线图，柱状图，饼图，雷达图等。在matplotlib的官网上给出了500多个图表展示的例子程序(<https://matplotlib.org/gallery/index.html>)。
- Matplotlib提供了详细的在线API文档(<https://matplotlib.org/api/>)，并且在每个函数介绍后提供了使用该函数的例子。

● 使用Matplotlib绘图



- **Matplotlib模块依赖于NumPy和tkinter模块，可以绘制多种形式的图形，包括线图、直方图、饼图、散点图等，图形质量满足出版要求，是数据可视化的重要工具。**
- **Matplotlib中应用最广的是matplotlib.pyplot模块。Pyplot提供了一套和Matlab类似的绘图API，使得Matplotlib的机制更像Matlab，只需要调用Pyplot模块所提供的函数就可以实现快速绘图并设置图表的各个细节。**
- **使用matplotlib时，使用的导入惯例为：**
`import matplotlib.pyplot as plt`

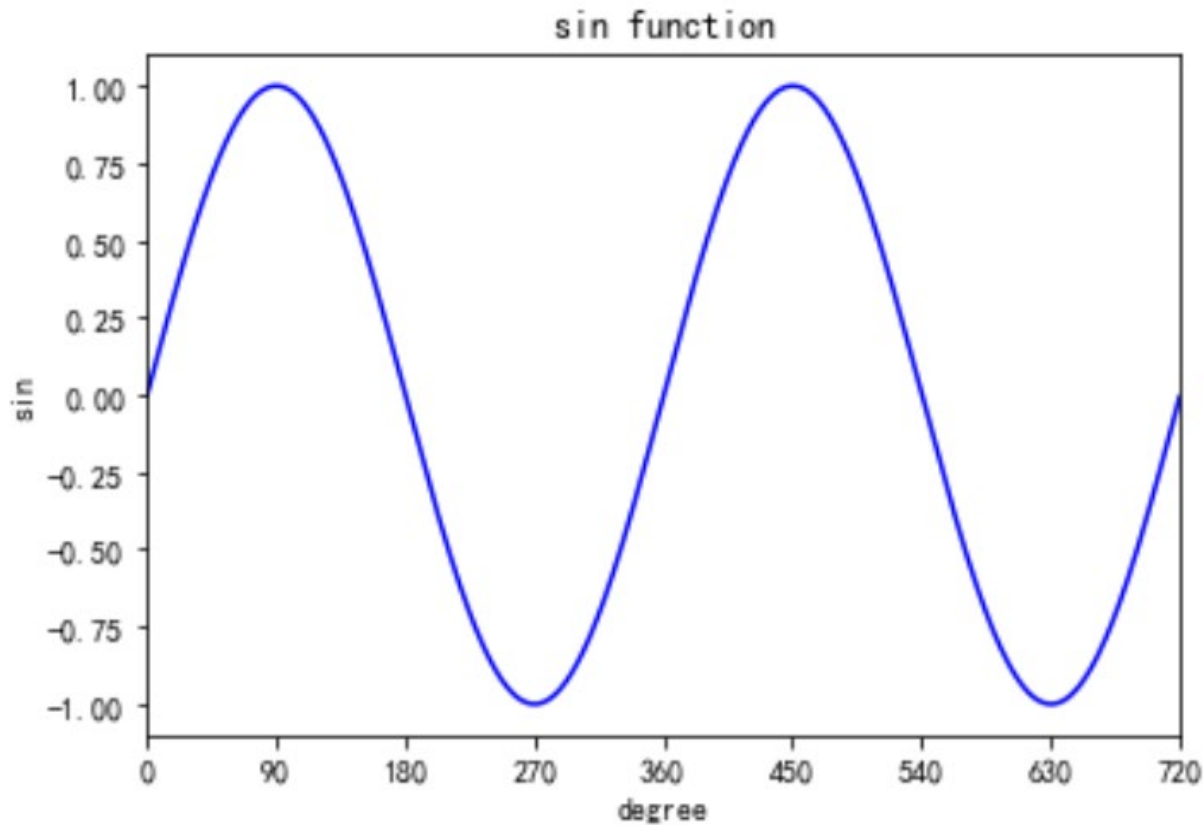
● 使用Matplotlib绘图



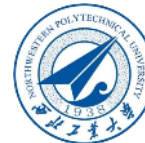
- **matplotlib**可绘制不同的图表，下面以一个简单的正弦函数为例说明图表的绘制方法。
- 程序首先准备了x, y两个数组，x是0~720度，y是对应的正弦函数值。使用**axis**函数设定x, y坐标轴的起止范围，**plot**函数画出正弦曲线，**xticks**指定x轴的小刻度标示。**xlabel**和**ylabel**分别是x, y轴的主题，**title**设置了图表的主题。

```
import matplotlib.pyplot as plt
import math
x=range(721)
y=[math.sin(i*3.14/180) for i in x]
plt.axis([0, 720, -1.1, 1.1])
plt.plot(x, y, 'b')
plt.xticks(range(0, 721, 90))
plt.xlabel("degree")
plt.ylabel("sin")
plt.title("sin function")
plt.show()
```


● 使用Matplotlib绘图

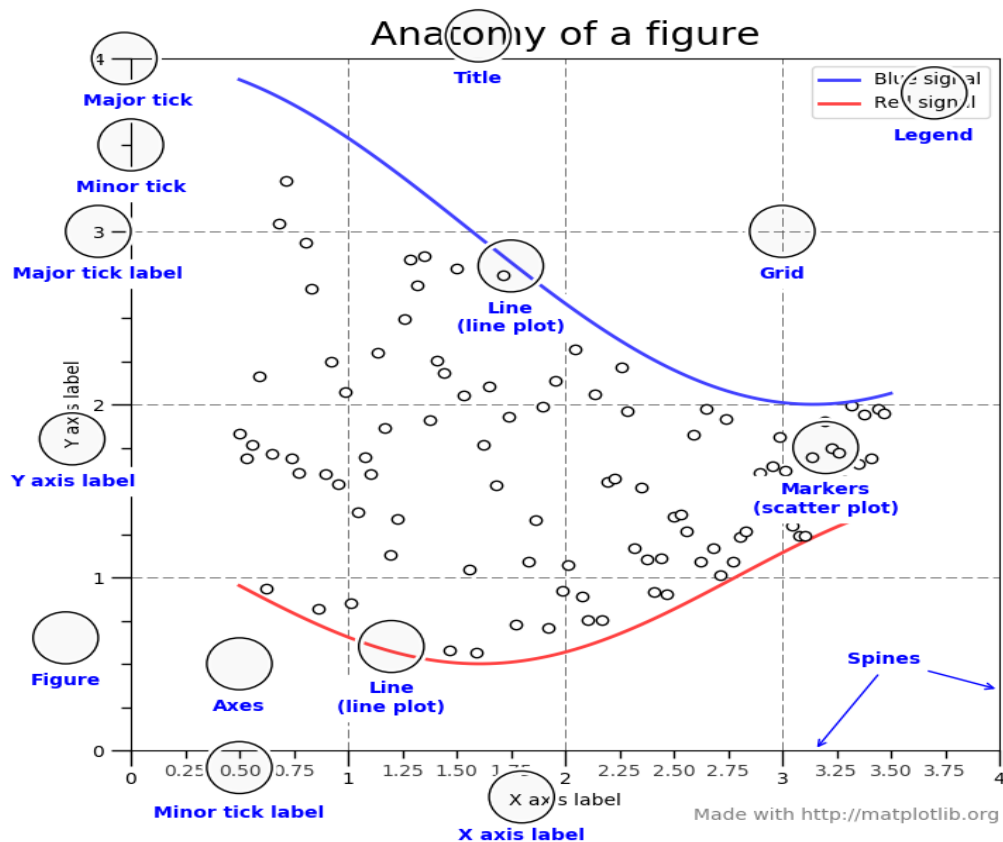


● 使用Matplotlib绘图



➤ 图表相关的术语:

一个图表中包含着很多图表元素，这些图表元素都有自己的名称和专业术语，只有知道了这些术语，才可以更好地编辑加工图表。图表区表示整个图表，包含所有数据系列、坐标轴、标题、图例、数据表等。绘图区是图表的一部分，由垂直坐标轴和水平坐标轴及其负轴包围的区域。右图给出了图表中的常用术语：



● 使用Matplotlib绘图



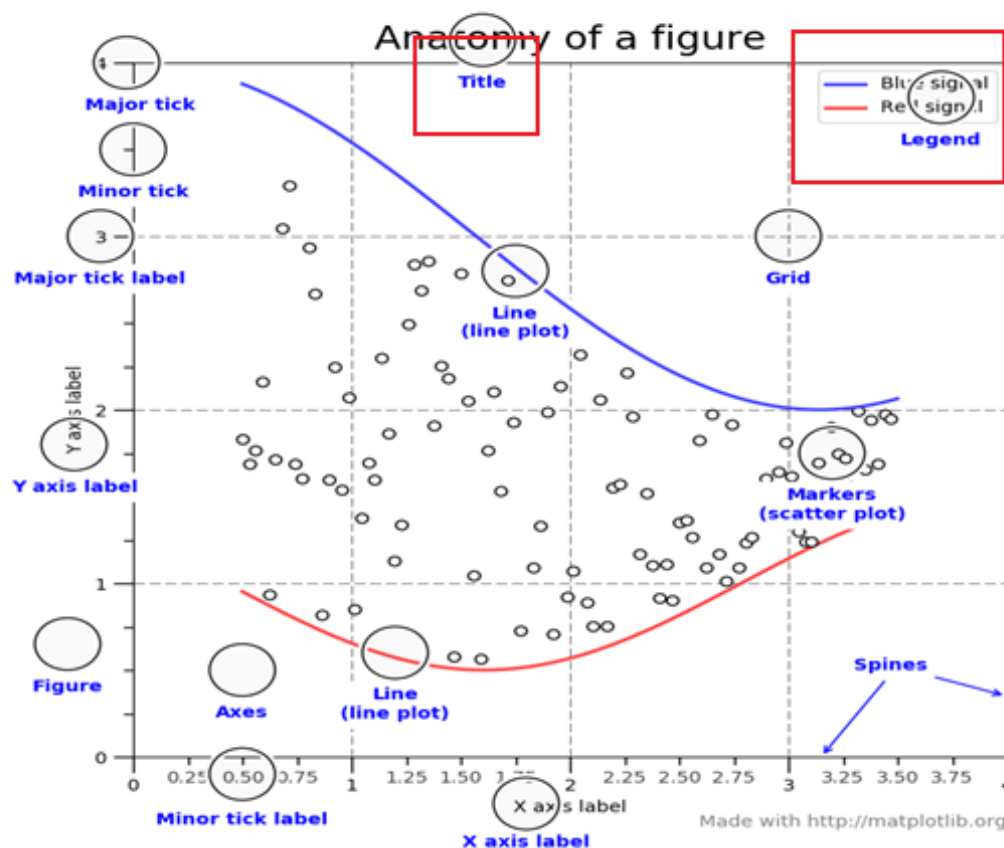
➤ 标题(Title)和图例(Legend)

➤ 图表标题 (Title) 用于对图表的功能进行说明 通常出现在

在图表区的顶
识为图表中每
况下，图例放

➤ 数据标签是
上的显示是可
名称”、“类
寸”。text函数

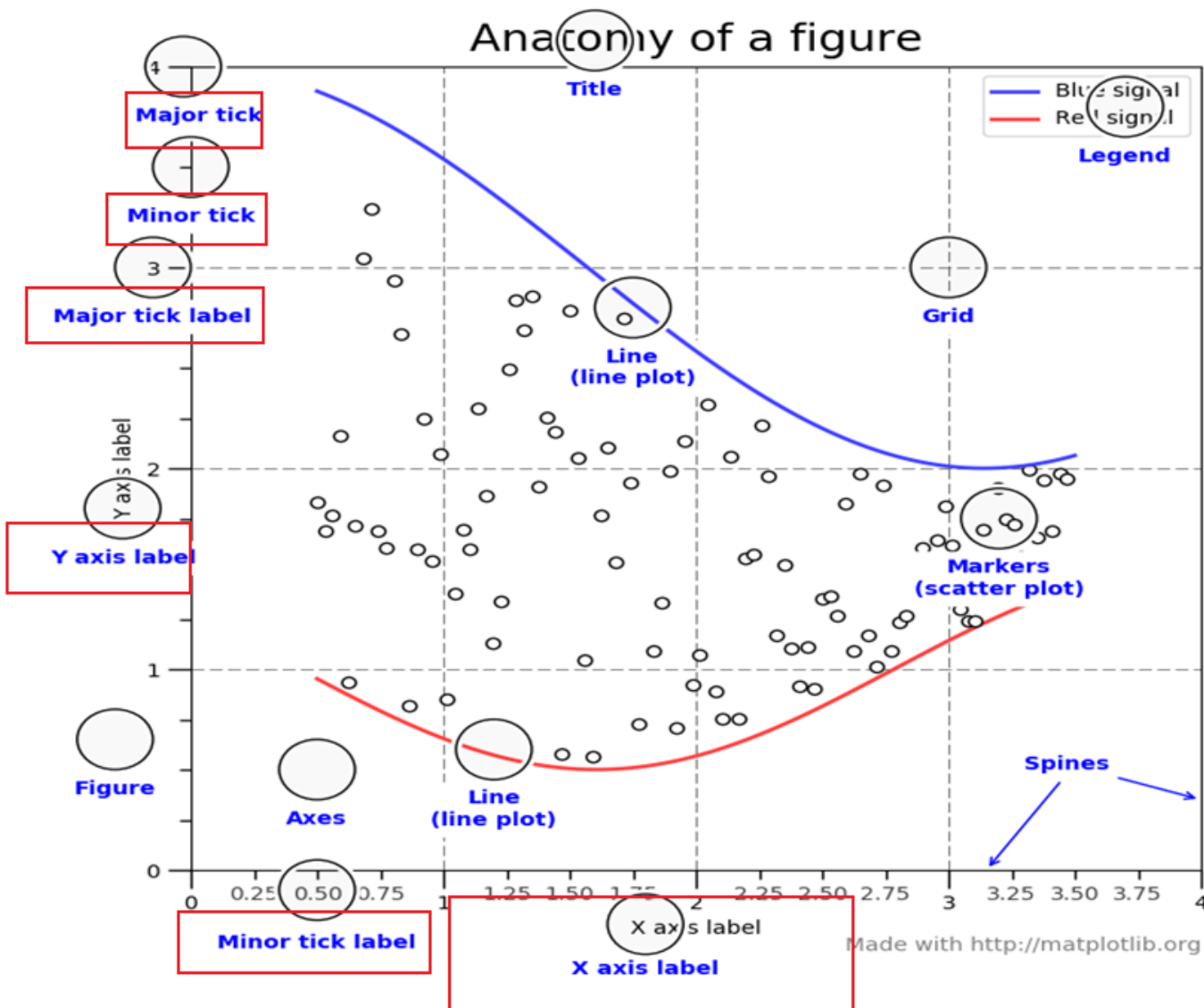
标
人情
表
系列
已



● 使用Matplotlib绘图



- 坐标轴
- 一般情况
类轴（或 / 和
三个轴。饼图
- 某些组合
数值轴绘图
要各种不同
数值轴在绘
数值轴在绘
- 坐标轴
标题（label
相交。刻度
进行说明的



➤ 数据

➤ 数据在绘图区通过不同的函数可以表现为线 (plot)，柱状图 (bar)，散点 (scatter) 等。绘制折线图时，每条折线就是一个数据系列，对应的y值是一个列表，和x的列表大小一致。图表中的数据序列可以指定颜色或图案，并展示在图表的图例中。折线图，散点图可以有一个或多个数据序列。但是饼图只有一个数据序列。

● 使用Matplotlib绘图



➤ 网

➤ 网

据的

区。

称为

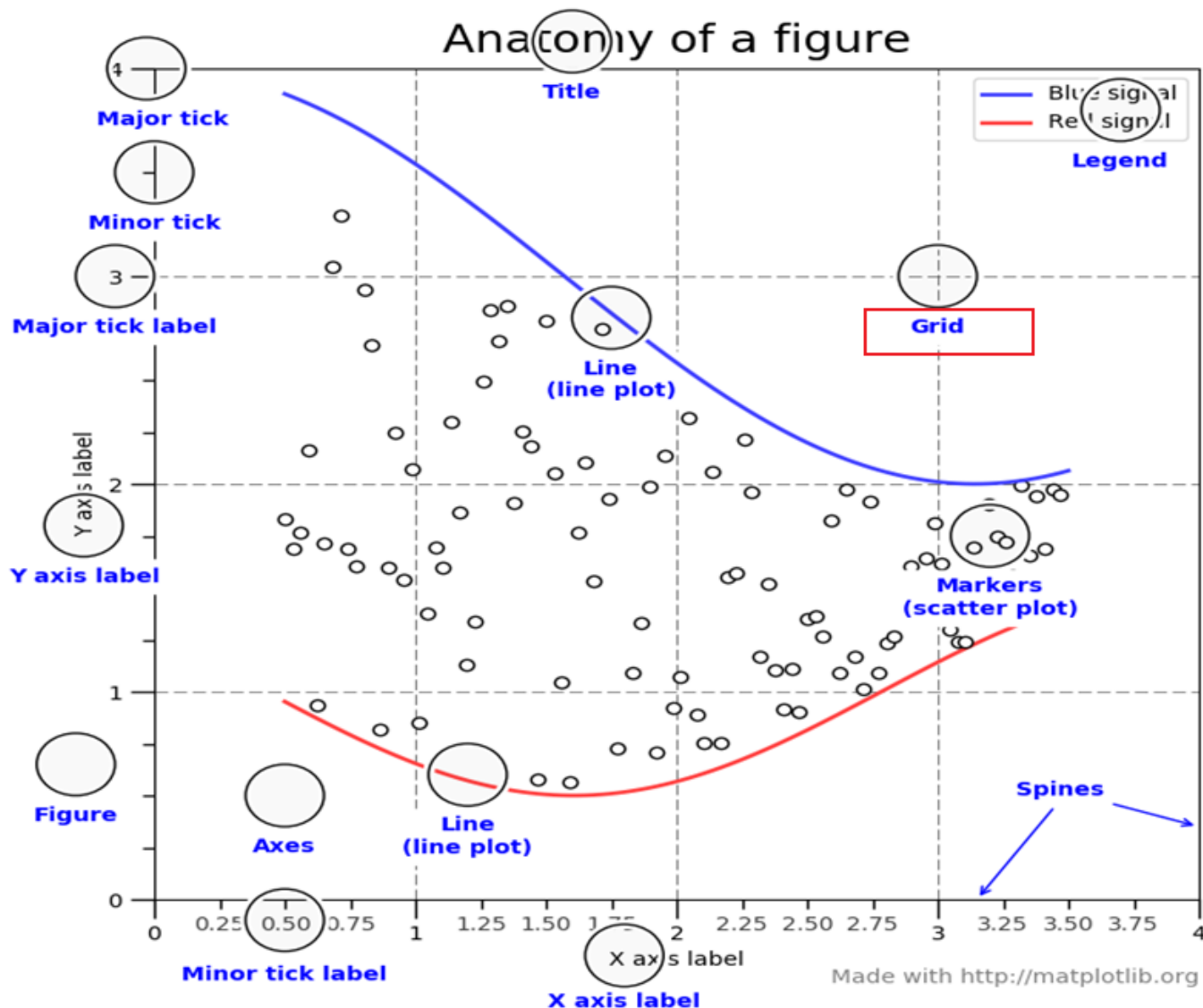
可参

➤ 趋

们常

scik

数用

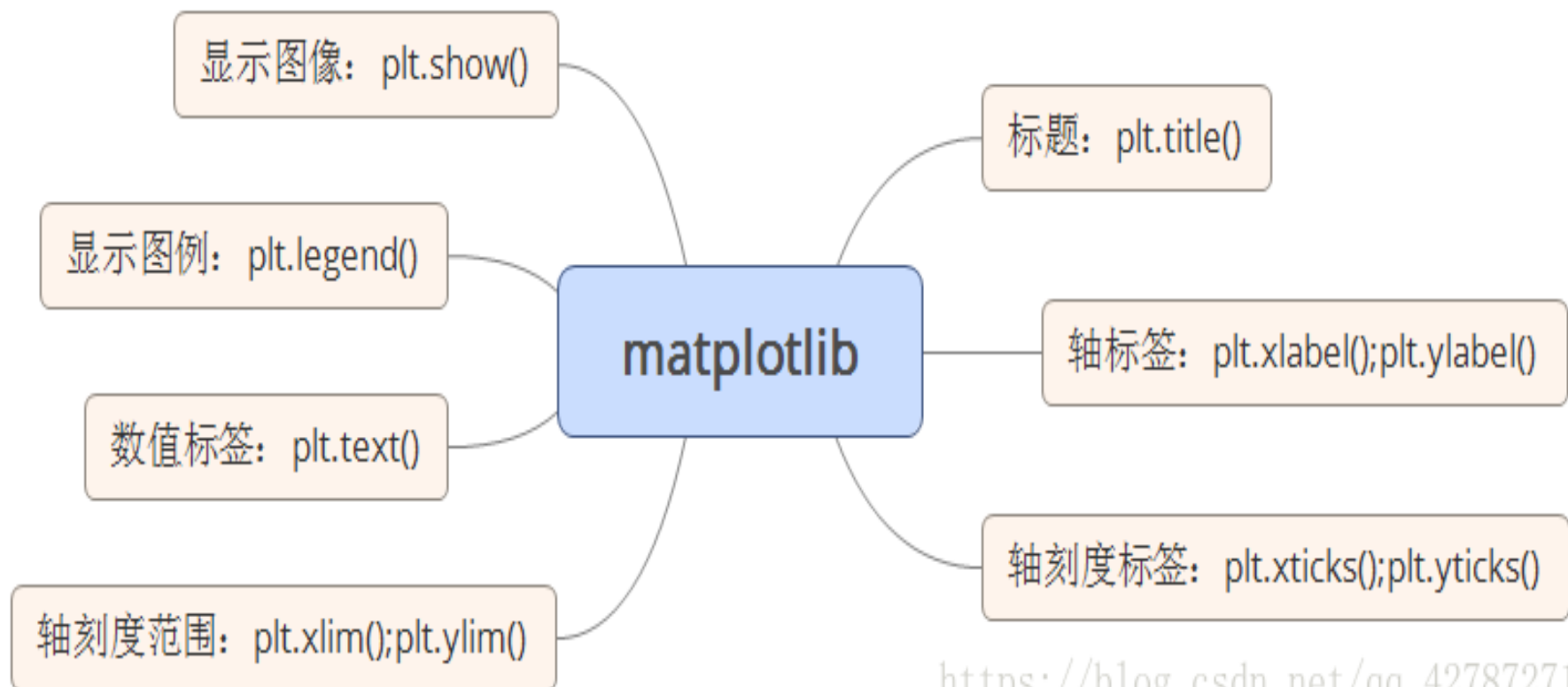


数
图
也
，
它
y，
函

● 使用Matplotlib绘图



➤ Matplotlib绘图基础



➤ 相关函数简介

`figure()`：创建一个新的绘图窗口。

`figtext()`：为figure添加文字

`axes()`：为当前figure添加一个坐标轴

`plot()`：绘图函数

`polar()`：绘制极坐标图

`axis()`：获取或设置轴属性的边界方法（坐标的取值范围）

`clf`：清除当前figure窗口

`cla`：清除当前axes窗口

`close`：关闭当前figure窗口

`subplot`：一个图中包含多个axes

`text()`：在轴上添加文字

➤ 相关函数简介

`title()`：设置当前axes标题

`xlabel/ylabel`：设置当前X轴或Y轴的标签

`hist()`：绘制直方图

`hist2d()`：绘制二维直方图

`hold`：设置当前图窗状态；off或者on

`imread()`：读取一个图像，从图形文件中提取数组

`legend()`：为当前axes放置标签

`pie()`：绘制饼状图

`scatter()`：做一个X和Y的散点图，其中X和Y是相同长度的序列对象

`stackplot()`：绘制一个堆叠面积图

➤ 相关函数简介

`acorr()`：绘制X的自相关函数

`annotate()`：用箭头在指定的数据点创建一个注释或一段文本

`bar()`：绘制垂直条形图

`barh()`：绘制横向条形图

`barbs()`：绘制一个倒钩的二维场

➤ 图表类型及相关函数

图表类型	函数	备注
折线图	<code>plot(x, y, format_string, **kwargs)</code>	二维线画图函数，可以绘制折线图和散点图
柱状图	<code>bar(x, height, width, *, align='center', **kwargs)</code>	绘制柱状图，该函数可以多次调用，有叠加效果。
散点图	<code>scatter(x, y)</code>	创建一个包含圆形的散点图，x，y是长度相同数组。

➤ 图表类型及相关函数

图表类型	函数	备注
饼图	<code>pie()</code>	绘制饼状图，可以自己计算每个类别所占的比例。
直方图	<code>hist()</code>	输入一个序列，并指定分为几个区间进行统计。
面积图	<code>stackplot(x, *args, **kwargs)</code>	绘制面积图，图像形如在不同折线间填充了颜色。

➤ 坐标轴格及相关函数

坐标轴格式	函数	备注
X轴标题	<code>xlabel()</code>	设置图片中X轴的标签信息
X轴刻度线	<code>xticks(x_tick,xtick_label,fontproperties,fontsize)</code>	设置X轴的刻度信息（允许使用文本作为刻度值）
X轴坐标起止范围	<code>xlim(xmin,xmax)</code>	设置X轴坐标的起止范围
Y轴标题	<code>ylabel()</code>	设置图片中Y轴的标签信息
Y轴刻度线	<code>yticks()</code>	设置Y轴的刻度信息（允许使用文本作为刻度值）
Y轴坐标起止范围	<code>ylim(ymin,ymax)</code>	设置Y轴坐标的起止范围
坐标起止范围	<code>axis([xmin xmax ymin ymax])</code>	绘图中用于设置坐标值范围

➤ 标注类型及相关函数

标注类型	函数	备注
图表标题	<code>title()</code>	设置整个数据图的标题
图例	<code>legend(*args, **kwargs)</code>	添加图例
数据标签	<code>text(x, y, s)</code>	可以在绘制的图片任何地方添加描述信息
箭头	<code>arrow()</code>	绘制一条带箭头的直线

➤ 其它一些功能

功能	函数	备注
隐藏图表	<code>hide()</code>	隐藏图表
显示图表	<code>show()</code>	将图形呈现出来
增加子图	<code>add_subplot(*args, **kwargs)</code>	增加子图
创建多个子图	<code>subplots(3,1,1)</code>	该函数的第一个参数是子图的行数，第二个参数是子图的列数，第三个参数是一个从1开始的序号

➤ 散点图

散点图由一些不连续的点组成，用来研究两个变量的相关性。包括正相关、负相关、不相关。主要通过scatter函数画散点图。

```
def scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None,
            vmin=None, vmax=None, alpha=None, linewidths=None, verts=None,
            edgecolors=None, hold=None, data=None, **kwargs):
```

主要参数：

x/y：数据

都是向量，而且必须长度相等。

s：标记大小

以像素点的平方为单位的标记面积，指定为下列形式之一：

- 数值标量：以相同的大小绘制所有标记。
- 行或列向量：使每个标记具有不同的大小。x、y 和 sz向量中的相应元素确定每个标记的位置和面积。sz向量的长度必须等于 x 和 y 的长度。
- 默认大小为rcParams['lines.markersize'] ** 2。

➤ 散点图

c: 标记颜色

标记颜色，指定为下列形式之一：

- **RGB** 三元数或颜色名称 - 使用相同的颜色绘制所有标记。
- 由**RGB**三元数组成的三列矩阵 - 对每个标记使用不同的颜色。

矩阵的每行为对应标记指定一种 **RGB** 三元数颜色。行数必须等于 **x** 和 **y** 的长度。

• 向量 - 对每个标记使用不同的颜色，并以线性方式将 **c** 中的 值映射到当前颜色图中的颜色。**c** 的长度必须等于 **x** 和 **y** 的长度。 要更改坐标区的颜色图，需使用 **colormap** 函数。

在绘制散点图时，如果希望散点颜色成为颜色图的索引，可以使用 **RGB**颜色对照表中色值代码的形式指定 **c**。

➤ 颜色对照表

选项	说明	对应的RGB色值代码
'red' 或 'r'	红色	#FF0000
'green' 或 'g'	绿色	#00FF00
'blue' 或 'b'	蓝色	#0000FF
'yellow' 或 'y'	黄色	#FFFF00
'magenta' 或 'm'	品红色	#FF00FF
'cyan' 或 'c'	青蓝色	#00FFFF
'white' 或 'w'	白色	#FFFFFF
'black' 或 'k'	黑色	#000000

注意：在绘制散点图时，如果要使用非预定义颜色绘制点，也可以使用RGB颜色对照表中色值代码的形式来指定颜色，形如 `c=["#FFDAB9","#FF1493","#E066F"]`，该方法可以指定任意颜色。使用时要注意c中参数个数与x，y个数相等。

● 使用Matplotlib绘图



➤ **marker:** 标记样式，其中值和说明如表所示。

值	说明
'o'	圆圈
'+'	加号
'*'	星号
'.'	点
'x'	叉号
'square' 或 's'	方形
'diamond' 或 'd'	菱形
'^'	上三角

➤ 散点图

edgecolors: 轮廓颜色

和c类似，参数也相同

alpha: 透明度

[0,1]: 1不透明，0透明

cmap: 色彩盘

可以使用默认的也可以使用自定义的，它实际上就是一个三列的矩阵(或者说，shape为 [N, 3]的 array)

- 矩阵中的值取值范围为 [0., 1.]
- 每一行代表一个颜色 (RGB)

linewidths: 线宽

标记边缘的宽度，默认是没有外围轮廓线。

注意事项:

color、marker等不能同时作为一个参数，`plt.scatter(x1, y1, 'bo', s=5)`不合法。

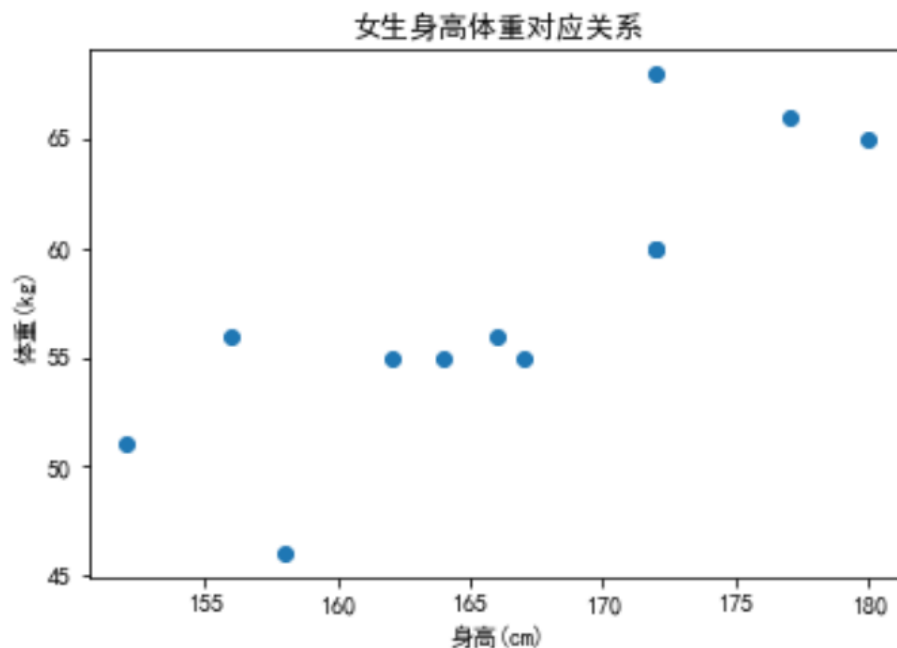
➤ 散点图

比如现在有一组女大学生的身高和体重数据，可以画散点图来观察两组数据之间的关系。

编号	1	2	3	4	5	6	7	8	9	10	11	12
身高(cm)	152	158	167	172	162	166	180	177	172	172	156	164
体重(kg)	51	46	55	60	55	56	65	66	68	60	56	55

➤ 散点图

```
import matplotlib.pyplot as plt
import numpy as np
#用来正常显示中文标签
plt.rcParams['font.family']='SimHei']
height=[152,158,167,172,162,166,180,177,172,172,156,164]
weight=[51,46,55,60,55,56,65,66,68,60,56,55]
plt.scatter(height,weight)
plt.xlabel("身高(cm)")
plt.ylabel("体重(kg)")
plt.title("女生身高体重对应关系")
plt.show()
```



➤ 散点图

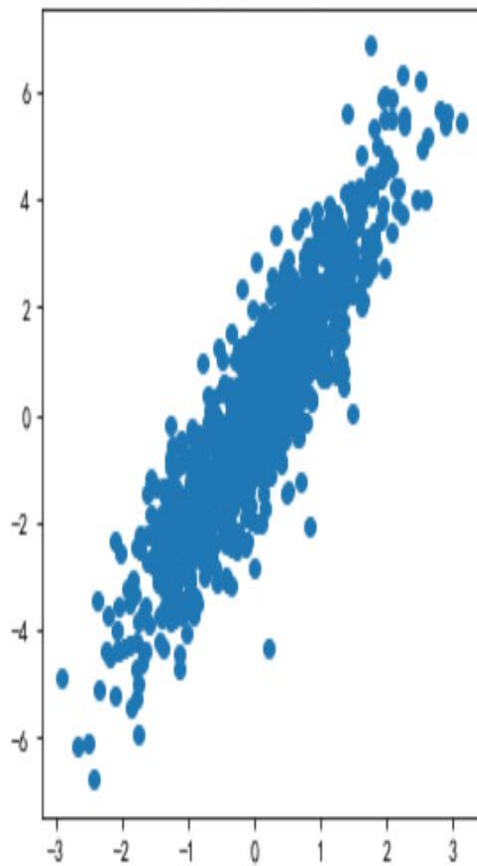
接下来通过例子分别展示正相关，负相关，不相关。首先使用NumPy的随机函数生成1000个随机数 x 。 y_1 ， y_2 分别为 $2x$ +随机数， $-2x$ -随机数，则 y_1 与 x 为正相关， y_2 与 x 为负相关。 y_3 也采用NumPy的随机函数生成1000个随机数，则 y_3 与 x 不相关。

➤ 散点图

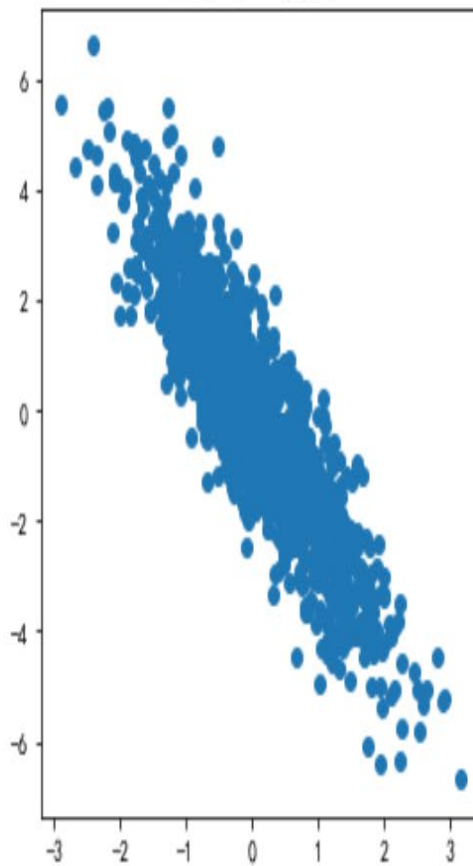
```
▶ import matplotlib.pyplot as plt
import numpy as np
#用来正常显示中文标签
plt.rcParams['font.family']=['SimHei']
# 解决保存图像是负号 '-' 显示为方块的问题
plt.rcParams['axes.unicode_minus'] = False
x=np.random.randn(1000)
y1=2*x+np.random.randn(1000)
y2=-2*x+np.random.randn(1000)
y3=np.random.randn(1000)
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.scatter(x, y1)
plt.title('散点图示例-正相关')
plt.subplot(1, 3, 2)
plt.scatter(x, y2)
plt.title('散点图示例-负相关')
plt.subplot(1, 3, 3)
plt.scatter(x, y3)
plt.title('散点图示例-不相关')
plt.show()
```


➤ 散点图

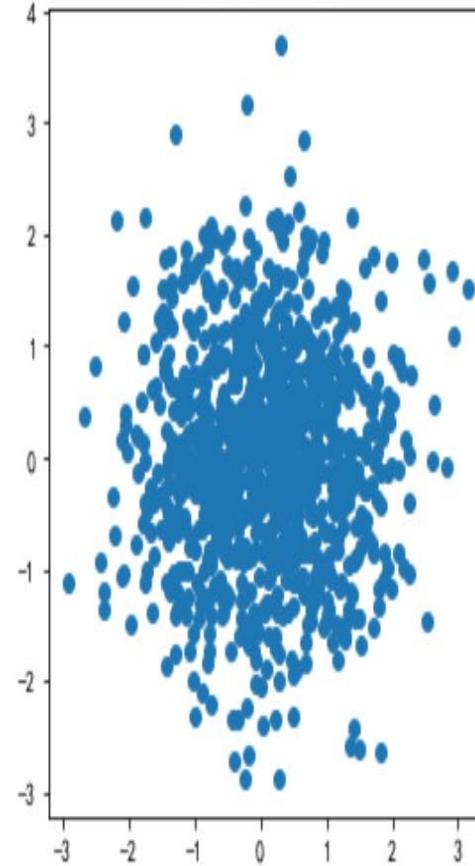
散点图示例-正相关



散点图示例-负相关



散点图示例-不相关



➤ 散点图

散点图有很多参数，最常用的是颜色，点大小，透明度，点形状。参数中s是指点的面积大小，c是点的颜色，marker是指点的形状，alpha是指点的透明度。散点图中透明度是一个很好的参数，可以看出哪些数据比较集中，哪些不集中。在大数据处理的聚类分析中，常用不同颜色或形状的点表示不同类别的数据，可以得到非常直观的印象。

➤ 折线图

➤ 折线图常用于显示数据序列随时间变化，数据变化的趋势。非常适用于显示在相等时间间隔下数据的走向变化。主要通过plot函数来画折线图。

```
def plot(*args, **kwargs):
```

➤ 主要参数

➤ 其中参数args是一个可变长度参数，允许多个x，y对和一个可选的格式字符串。例如，以下各项都是合法的：

```
plot(x, y)      # plot x和y使用默认线条样式和颜色  
plot(x, y, 'bo') # plot x和y使用蓝色圆圈标记  
plot(y)         # plot y使用x作为索引数组0..N-1  
plot(y, 'r+')   # 同上，但是用红色加号
```

➤ 折线图

如果x和/或y是二维的，则将绘制相应的列。

x, y: 接收值为array, 表示 x 轴与 y 轴对应的数据；无默认值；

其它的参数用来控制颜色，线型，标注等。

(1) 针对线条的处理

- 线条类型

参数: linestyle或者ls, 表示折线的类型, 可以是实线、虚线、点虚线、点点线等, 即'-' , '--' , '-.' , ':' 等。

- 线条粗细

参数: linewidth或lw, 可自行设置, 默认值为1;

- 线条颜色

参数: color或c , 设置方法和散点图中设置颜色方法一致。

➤ 折线图

(2) 针对数据标记的处理

- 参数marker: 数据标记的类型
- 参数markeredgewidth 或 mew: 数据标记的宽度
- 参数markeredgecolor 或 mec: 数据标记的边界颜色
- 参数markerfacecolor 或 mfc: 数据标记的填充色
- alpha: 接收值为0~1之间的小数, 表示点的透明度;

(3) 图例

label: 表示数据图例内容。参数中label= ‘数据序列名称’ 指定该数据序列的名称, 其后通过调用legend() 函数可显示不同数据序列的图例。

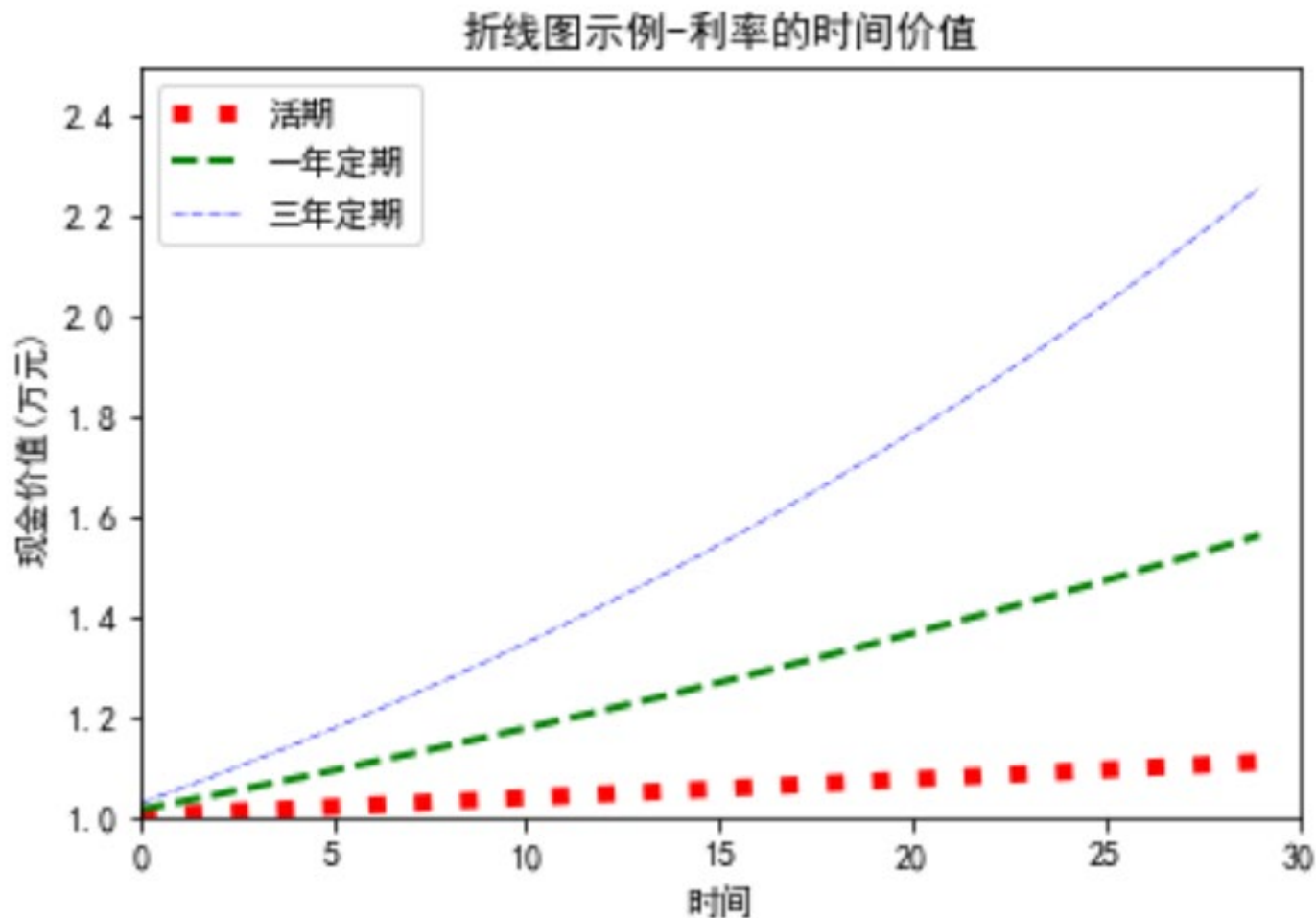
➤ 折线图

折线图中可以画多条折线表示不同的数据序列。下面以1万元分别存入活期，1年定期，3年定期后，在30年中的价值变化。2019年的活期，1年定期，3年定期的利率分别是0.35%，1.50%和2.75%。

➤ 折线图

```
▶ import matplotlib.pyplot as plt
#用来正常显示中文标签
plt.rcParams['font.family']=['SimHei']
x=range(30)
#活期利率0.35%
huoqi=[1.0035**(i+1) for i in x]
#一年定期利率1.5%
dingqi1=[1.015**(i+1) for i in x]
#活期利率2.75%
dingqi3=[1.0275**(i+1) for i in x]
#color指定线条颜色, label标签内容
plt.plot(huoqi,color='r',linewidth=5,linestyle=':',label='活期')
#linewidth指定线条粗细
plt.plot(dingqi1,color='g',linewidth=2,linestyle='--',label='一年定期')
#linestyle指定线形为点
plt.plot(dingqi3,color='b',linewidth=0.5,linestyle='-.',label='三年定期')
#标签展示位置, 数字代表标签具体位置
plt.legend(loc=2)
plt.axis([0,30,1,2.5])
plt.xlabel('时间')
plt.ylabel('现金价值(万元)')
plt.title('折线图示例-利率的时间价值')
plt.show()
```

➤ 折线图



➤ 折线图

- 上面提到过，折线图一般用来观察数据随时间的变化趋势，那么如果横坐标是年/月/日格式的话，matplotlib提供了另外的函数`plot_date`来将横坐标调整为日期格式，此函数可以设置颜色，线形，点状等。当然，也可以在一个图上画多个折线图。

- 上面讨论了散点图和折线图，分别用到函数`plt.scatter`和`plt.plot`，但其实`plt.plot`也可以画出散点图，两者在画散点图时区别不大，但是当数据很大时，`plt.plot`的效率将大大高于`plt.scatter`。因为`plt.scatter`会对每个单独的散点进行渲染，会耗费很多的资源，而`plt.plot`中，散点之间彼此复制，所有点的配置只需进行一次即可。因此处理大数据时，`plt.plot`方法比`plt.scatter`方法好。

● 使用Matplotlib绘图



➤ 条形图

➤ 条形图用宽度相同的条形的高度来表示数据的多少，分为水平条形图和垂直条形图。条形图显示各个项目的比较情况。在水平条形图中，垂直坐标轴表示不同的类别，水平坐标表示不同类别对应的值。通过plt.bar将数据通过条形图展现出来。

```
def bar(left, height, width=0.8, bottom=None, hold=None, data=None, **kwargs)
```

➤ 主要参数

- left: x轴的位置序列，一般采用range函数产生一个序列，但是有时候可以是字符串；
- height: 表示条形图的高度，也就是y轴的数值；
- alpha: 表示柱形图的颜色透明度，默认值为1；
- width: 表示柱形图的宽度，默认值为0.8；
- color (facecolor) : 柱形图填充的颜色，默认为随机色；

➤ 条形图

- `edgecolor`: 图形边缘颜色;
- `label`: 解释每个图像代表的含义;
- `linewidth` (`linewidths` / `lw`): 边缘线的宽度, 默认值为1。
- 注意: `barh()` 函数与 `bar()` 函数的主要区别是: 在 `bar()` 函数中, `width` 这一参数代表的是柱子的宽度 (胖瘦), 而在 `barh()` 函数中 `width` 这一参数代表的是横向柱子的长度 (长短)。

➤ 条形图

以2019年各省人口数据为例，数据来源为中国国家统计局

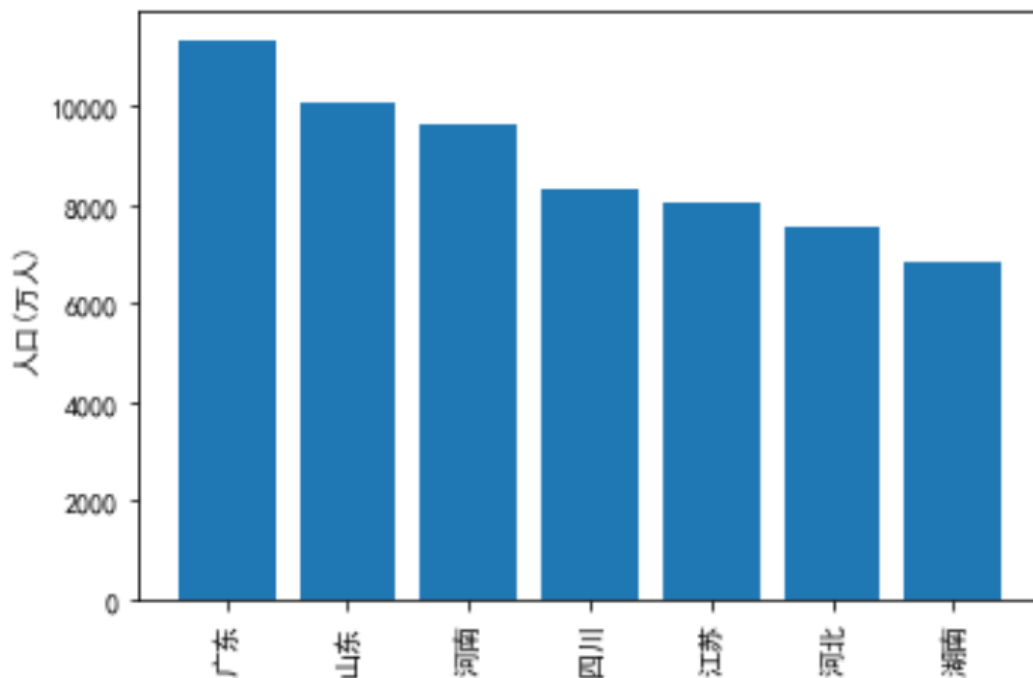
(<http://www.mnw.cn/news/shehui/726472.html>), 数据文件为“2019分省人口及GDP.xlsx”。因篇幅关系，省份和人口的数据没有全部列出来，图中只列举了7个省份的人口数据。

● 使用Matplotlib绘图



➤ 条形图

```
▶ import matplotlib.pyplot as plt
   #用来正常显示中文标签
   plt.rcParams['font.family']=['SimHei']
   province=['广东','山东','河南','四川','江苏','河北','湖南']
   population=[11346,10047.24,9605,8341,8029.3,7556.3,6860.2]
   plt.bar(range(len(province)),population)
   plt.xticks(range(len(province)),province,rotation=90)
   plt.ylabel("人口(万人)")
   plt.show()
```



➤ 条形图

● 有时显示在横轴上的数据很多，看起来很不方便。这种情况下可以采用水平条形图。第一种方法是在plt.bar函数中指定orientation='horizontal'参数，第二种方法是直接使用plt.barh函数。使用barh时x轴表示人口数量，y轴表示不同的省份。

```
plt.bar(0,bottom=range(len(province)),width=population,height=0.7,orientation='horizontal')
```

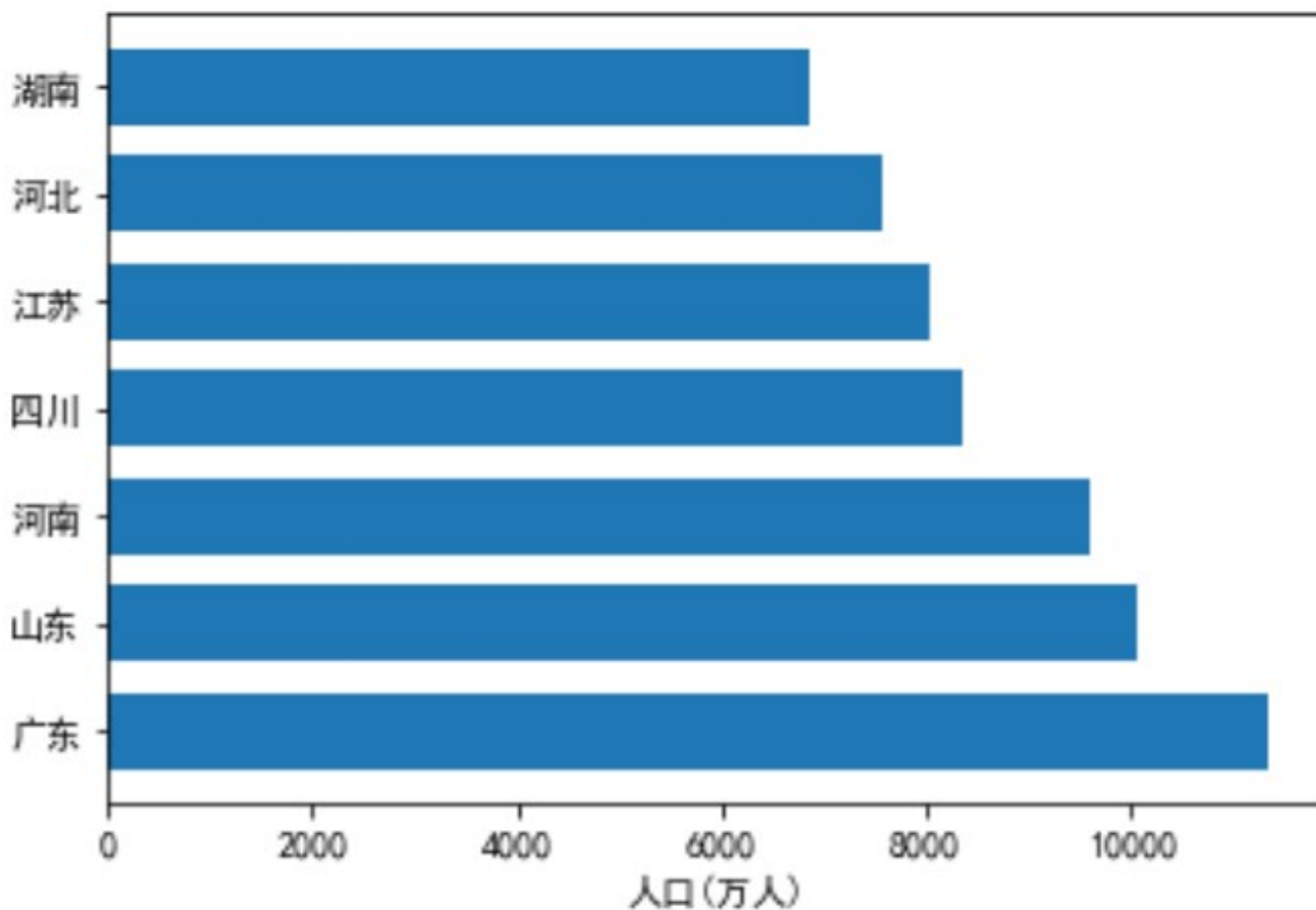
```
plt.barh(range(len(province)),population,height=0.7)
plt.yticks(range(len(province)),province)
plt.tick_params(labelsize=8)
plt.xlabel("人口(万人)")
```

➤ 条形图

```
plt.bar(0,bottom=range(len(province)),width=population,height=0.7,orientation='horizontal')
```

```
import matplotlib.pyplot as plt
#用来正常显示中文标签
plt.rcParams['font.family']=['SimHei']
province=['广东','山东','河南','四川','江苏','河北','湖南']
population=[11346,10047.24,9605,8341,8029.3,7556.3,6860.2]
plt.bar(0,bottom=range(len(province)),width=population,
        height=0.7,orientation='horizontal')
plt.yticks(range(len(province)),province)
plt.xlabel("人口(万人)")
plt.show()
```

➤ 条形图

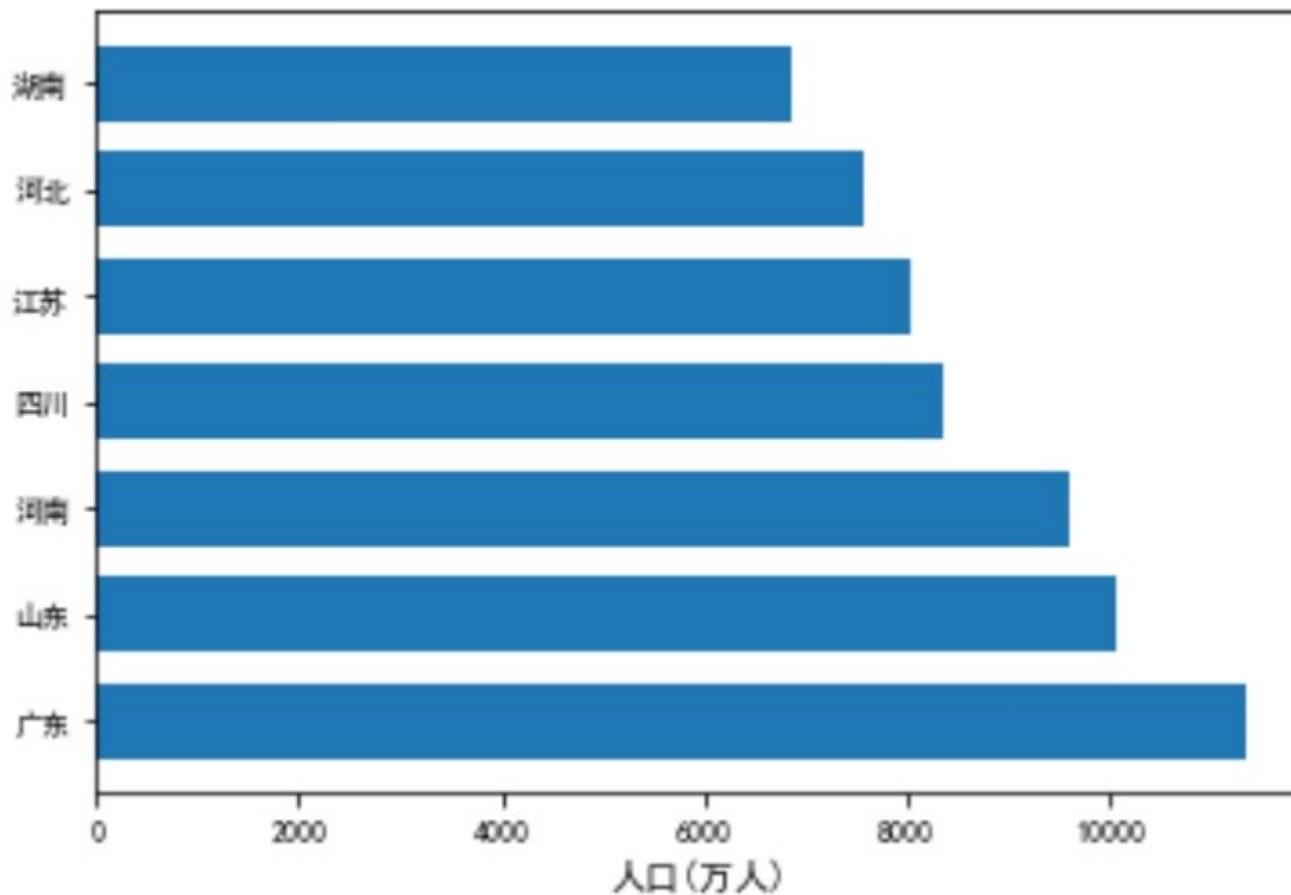


➤ 条形图

```
plt.barh(range(len(province)),population,height=0.7)
plt.yticks(range(len(province)),province)
plt.tick_params(labelsize=8)
plt.xlabel("人口(万人)")
```

```
❏ import matplotlib.pyplot as plt
   #用来正常显示中文标签
   plt.rcParams['font.family']=['SimHei']
   province=['广东','山东','河南','四川','江苏','河北','湖南']
   population=[11346,10047.24,9605,8341,8029.3,7556.3,6860.2]
   plt.barh(range(len(province)),population,height=0.7)
   plt.yticks(range(len(province)),province)
   plt.tick_params(labelsize=8)
   plt.xlabel("人口(万人)")
   plt.show()
```

➤ 条形图



➤ 条形图

- 当然，条形图不仅限于对比一组数据。常见的条形图还可以对比多组数据。使用并列条形图设置好不同系列的位置即可。下面是matplotlib官方的例子，展示了如何画出并列条形图。

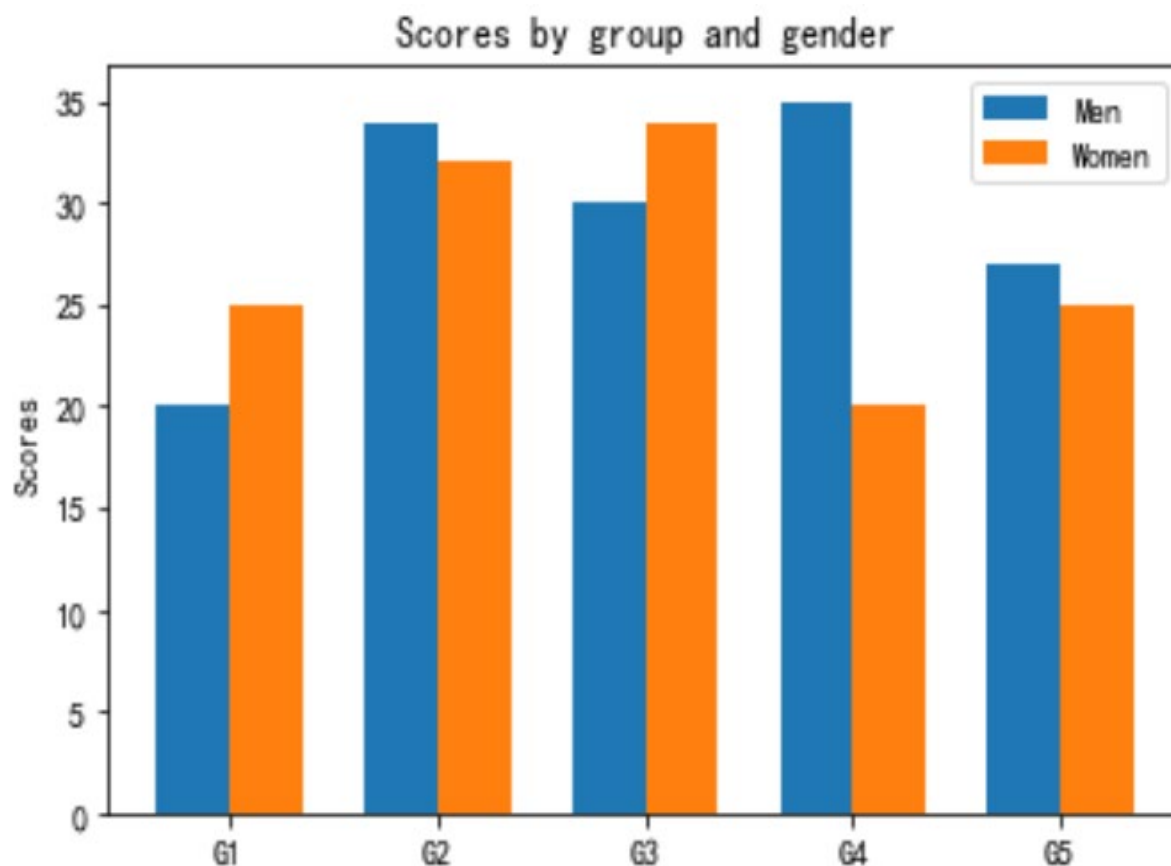
➤ 条形图

其中在调用bar时men_means对应的x为 $x - \text{width}/2$ ，而women_means对应的x为 $x + \text{width}/2$ 。由于指定了不同的x位置，两组数据可以并排放置。

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
labels = ['G1', 'G2', 'G3', 'G4', 'G5']
men_means = [20, 34, 30, 35, 27]
women_means = [25, 32, 34, 20, 25]
# the label locations
x = np.arange(len(labels))
# the width of the bars
width = 0.35
fig, ax = plt.subplots()
rects1 = ax.bar(x - width/2, men_means, width, label='Men')
rects2 = ax.bar(x + width/2, women_means, width, label='Women')
# Add some text for labels, title and custom x-axis tick labels, etc.
ax.set_ylabel('Scores')
ax.set_title('Scores by group and gender')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
```

➤ 条形图

其中在调用bar时men_means对应的x为 $x + \text{width}/2$ ，而women_means对应的x为 $x - \text{width}/2$ 。由于指定了不同的x位置，两组数据可以并排放置。



➤ 条形图

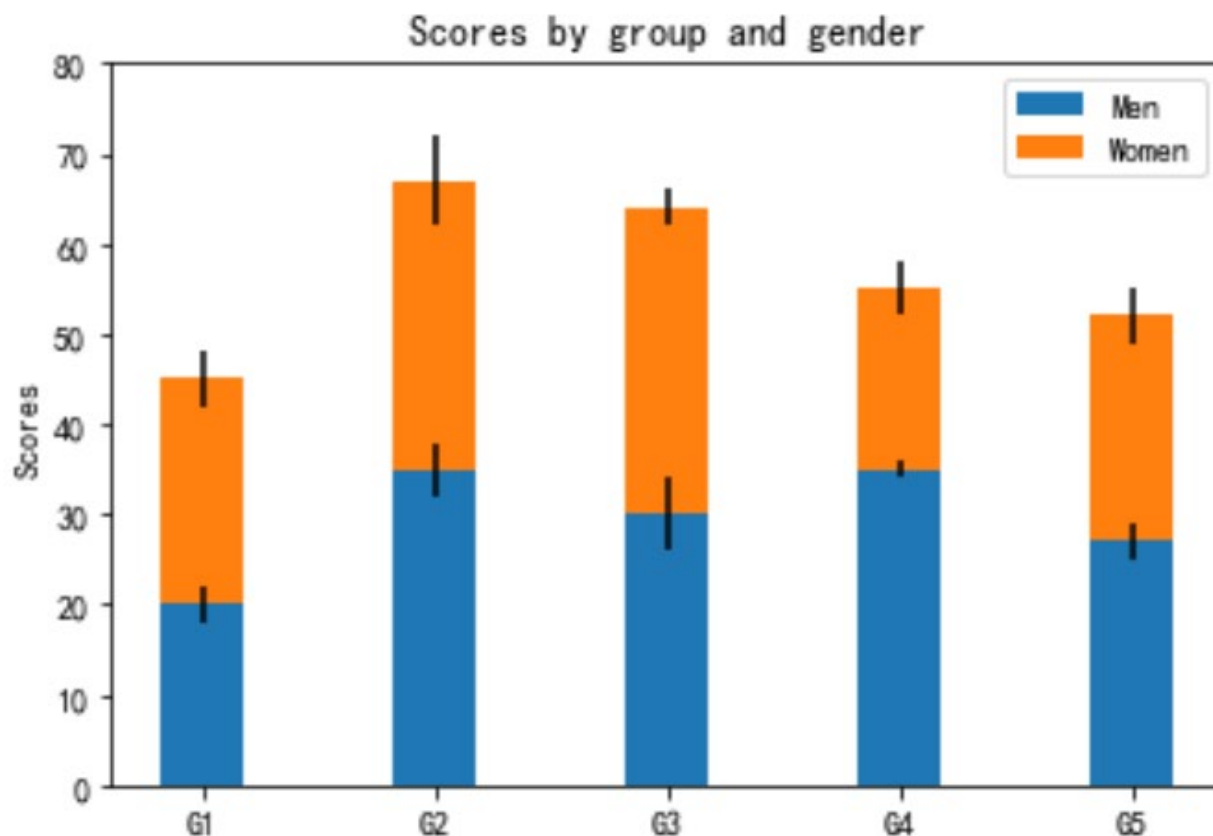
垂直堆叠条形图可以用来显示不同类别中各个子类的比例关系。如大学不同专业中男女生比例。垂直并列条形图需要通过bottom设置，bottom的坐标是建立在另一个条形图基础上。

➤ 条形图

```
import numpy as np
import matplotlib.pyplot as plt
N = 5
menMeans = (20, 35, 30, 35, 27)
womenMeans = (25, 32, 34, 20, 25)
#std values
menStd = (2, 3, 4, 1, 2)
womenStd = (3, 5, 2, 3, 3)
# the x locations for the groups
ind = np.arange(N)
# the width of the bars: can also be len(x) sequence
width = 0.35
#yerr set values of error
p1 = plt.bar(ind, menMeans, width, yerr=menStd)
p2 = plt.bar(ind, womenMeans, width, bottom=menMeans,
             yerr=womenStd)
plt.ylabel('Scores')
plt.title('Scores by group and gender')
plt.xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.yticks(np.arange(0, 81, 10))
plt.legend((p1[0], p2[0]), ('Men', 'Women'))
plt.show()
```

➤ 条形图

垂直并列条形图需要通过bottom设置，bottom的坐标是建立在另一个条形图基础上。例子中画womenMeans时，指定的bottom为menMeans，即女生的数据在男生的上方。



➤ 直方图

- 直方图(Histogram)由一系列高度不等的纵向条纹或线段表示数据分布的情况。 一般用横轴表示数据类型，纵轴表示分布情况。直方图是数值数据分布的精确图形表示。 这是一个连续变量（定量变量）的概率分布的估计，并且被卡尔·皮尔逊（Karl Pearson）首先引入。它是一种特殊的条形图，条形图中的数据一般是独立的，表示数据的矩形条之间一般有空白。而直方图一般用于统计不同区间的数据分布，表示数据的矩形条之间一般是连续的。

➤ 直方图

```
def hist(x,bins=None,range=None,normed=False,weights=None,cumulative=False,
        bottom=None, histtype='bar', align='mid', orientation='vertical',
        rwidth=None, log=False, color=None, label=None, stacked=False,
        hold=None, data=None, **kwargs):
```

➤ 主要参数：

- x: 数据集，最终的直方图将对数据集进行统计；
- bins: 指定直方图条形的个数；
- range: 显示的区间；
- normed: 标准化，是否将直方图的频数转换成频率；
- density: 显示的是频数统计结果，默认为false，为True则显示频率统计结果，这里需要注意，频率统计结果=区间数目/(总数*区间宽度)，和normed效果一致，官方推荐使用density；
- histtype: 指定直方图的类型，可选{'bar', 'barstacked', 'step', 'stepfilled'}之一，默认为bar，推荐使用默认配置，step使用的是梯状，stepfilled则会对梯状内部进行填充，效果与bar类似；

➤ 直方图

- align: 设置条形边界值的对其方式, 可选{'left', 'mid', 'right'}之一, 默认为'mid', 还有 left或者right, 会有部分空白区域, 推荐使用默认;
- orientation: 水平或垂直方向['horizontal', 'vertical'], 默认为垂直方向;
- rwidth: 柱子与柱子之间的距离, 默认是0;
- log: 是否需要将绘图数据进行log变换, 默认False;
- stacked: 当有多个数据时, 是否需要将直方图呈堆叠摆放, 默认水平摆放。
- color: 设置直方图颜色;
- label: 设置直方图的标签, 可通过legend展示其图例;
- bottom: 可以为直方图的每个条形添加基准线, 默认为0。
- hist函数主要输入一个序列, 并指定分为几个区间进行统计。按照在每个区间内的数据数量绘制图形, 展示不同区间的数据数量。

➤ 直方图

- 下面的例子展示了如何绘制直方图。

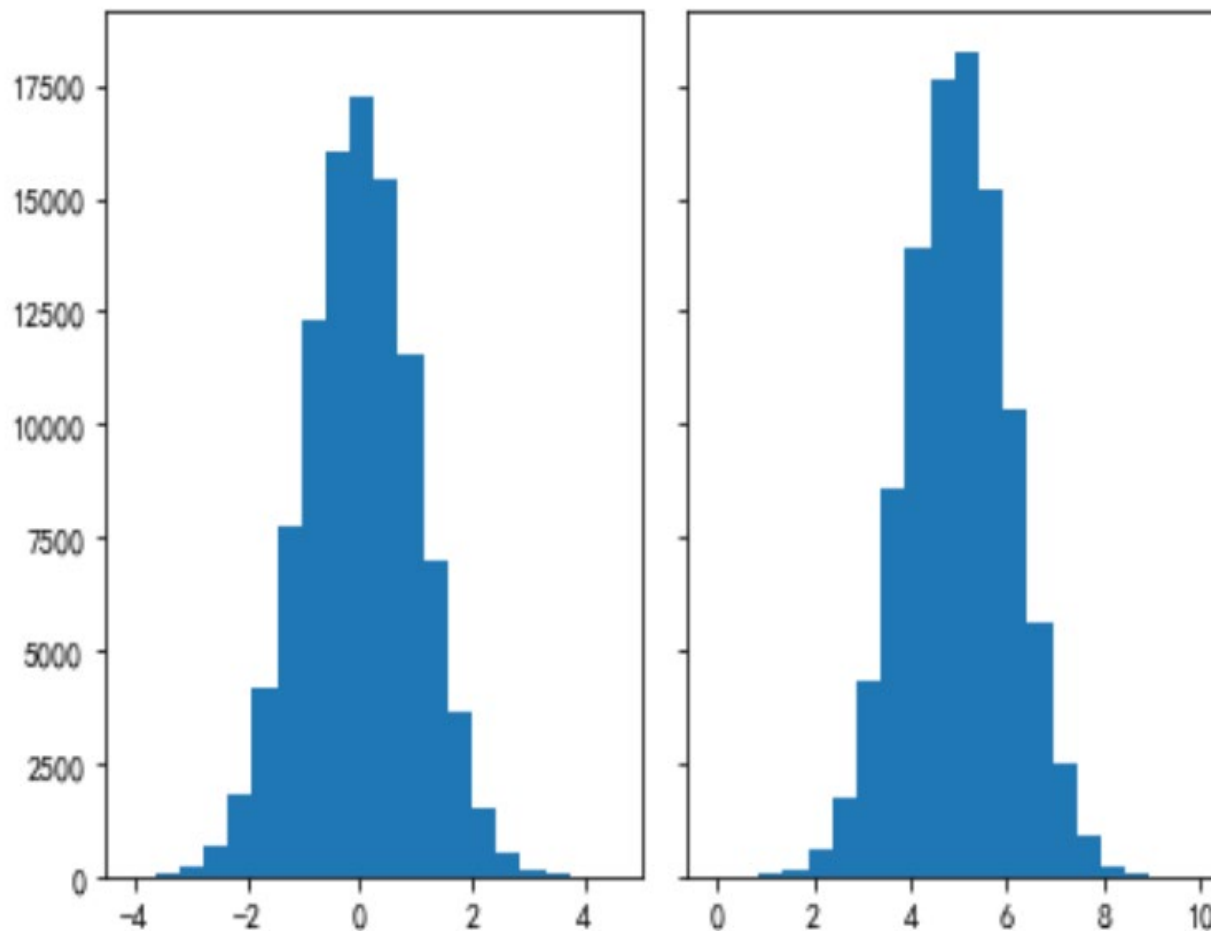
```
import matplotlib.pyplot as plt
import numpy as np
N_points = 100000
n_bins = 20
# Generate a normal distribution, center at x=0 and y=5
x = np.random.randn(N_points)
y = .4 * x + np.random.randn(100000) + 5
fig, axs = plt.subplots(1, 2, sharey=True, tight_layout=True)
# We can set the number of bins with the `bins` kwarg
axs[0].hist(x, bins=n_bins)
axs[1].hist(y, bins=n_bins)
plt.show()
```

● 使用Matplotlib绘图



➤ 直方图

- 下面的例子展示了如何绘制直方图。



➤ 面积图

● 面积图可用于绘制随时间发生的变化量，用于引起人们对总值趋势的关注。通过显示所绘制的值的总和，面积图还可以显示部分与整体的关系。使用`plt.stackplot`函数可绘制面积图。从函数名称也可以看出，面积图和折线图有些类似。从图的表现形式来看，面积图就像是在不同折线间填充了颜色。

```
def stackplot(x, *args, **kwargs):
```

➤ 主要参数：

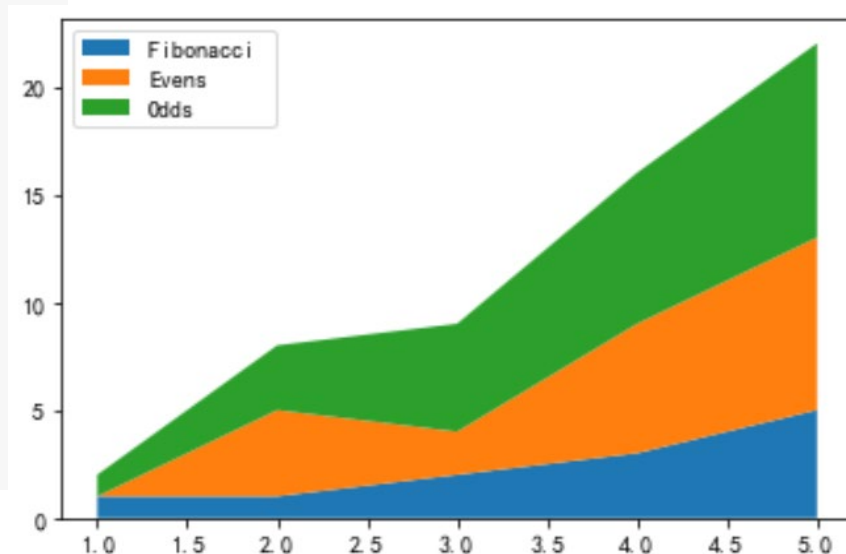
- **x**：指定面积图的x轴数据；
- ***args**：可变参数，可以接受任意多的y轴数据，即各个拆分的数据对象；
- ****kwargs**：关键字参数，可以通过传递其他参数来修饰面积图，如标签、颜色，用法与之前的**labels**、**colors**用法一致。

● 使用Matplotlib绘图



➤ 面积图

```
import numpy as np
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [1, 1, 2, 3, 5]
y2 = [0, 4, 2, 6, 8]
y3 = [1, 3, 5, 7, 9]
y = np.vstack([y1, y2, y3])
labels = ["Fibonacci ", "Evens", "Odds"]
fig, ax = plt.subplots()
ax.stackplot(x, y1, y2, y3, labels=labels)
ax.legend(loc='upper left')
plt.show()
```



➤ 饼图

- 饼图常用来对数据的比例进行展示，比如大学中来自不同省份学生的比例，每月家庭开支中各项内容所占比例。
- 主要通过pie函数来画饼图。

```
def pie(x, explode=None, labels=None, colors=None, autopct=None,
        pctdistance=0.6, shadow=False, labeldistance=1.1, startangle=None,
        radius=None, counterclock=True, wedgeprops=None, textprops=None,
        center=(0, 0), frame=False, hold=None, data=None):
```

➤ 主要参数：

- **x**: (每一块)的比例，如果 $\text{sum}(x) > 1$ 会使用 $\text{sum}(x)$ 归一化；
- **labels**: (每一块)饼图外侧显示的说明文字；
- **explode**: (每一块)离开中心距离；
- **startangle**: 起始绘制角度，默认图是从x轴正方向逆时针画起，如设定=90则从y轴正方向画起；

➤ 饼图

- shadow: 在饼图下面画一个阴影。默认值: False, 即不画阴影;
- labeldistance: label标记的绘制位置, 相对于半径的比例, 默认值为1.1, 如<1则绘制在饼图内侧;
- autopct: 控制饼图内百分比设置, 可以使用format字符串或者format function
- '%1.1f' 指小数点前后位数(没有用空格补齐);
- pctdistance: 类似于labeldistance, 指定autopct的位置刻度, 默认值为0.6;
- radius: 控制饼图半径, 默认值为1;
- counterclock: 指定指针方向; 布尔值, 可选参数, 默认为: True, 即逆时针。将值改为False即可改为顺时针。
- wedgeprops: 字典类型, 可选参数, 默认值: None。参数字典传递给wedge对象用来画一个饼图。例如: wedgeprops={'linewidth':3} 设置wedge线宽为3。

➤ 饼图

- `textprops`: 设置标签 (labels) 和比例文字的格式; 字典类型, 可选参数, 默认值为: `None`。传递给`text`对象的字典参数。
- `center`: 浮点类型的列表, 可选参数, 默认值: `(0, 0)`, 图标中心位置。
- `frame`: 布尔类型, 可选参数, 默认值: `False`。如果是`true`, 绘制带有表的轴框架。
- `rotatelabels`: 布尔类型, 可选参数, 默认为: `False`。如果为`True`, 旋转每个`label`到指定的角度。

● 使用Matplotlib绘图



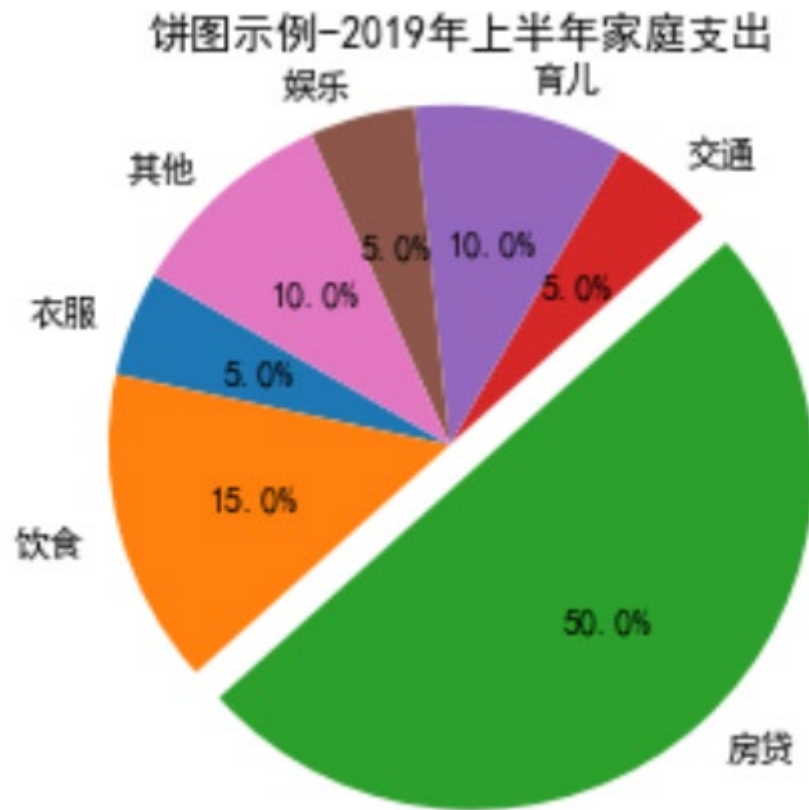
➤ 饼图

```
import matplotlib.pyplot as plt
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
labels = ['衣服', '饮食', '房贷', '交通', '育儿', '娱乐', '其他']
sizes = [5, 15, 50, 5, 10, 5, 10]
explode = (0, 0, 0.1, 0, 0, 0, 0)
plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=False, startangle=150)
plt.title("饼图示例-2019年上半年家庭支出")
#将饼图显示为正圆形
plt.axis('equal')
plt.show()
```

● 使用Matplotlib绘图



➤ 饼图



➤ 子图

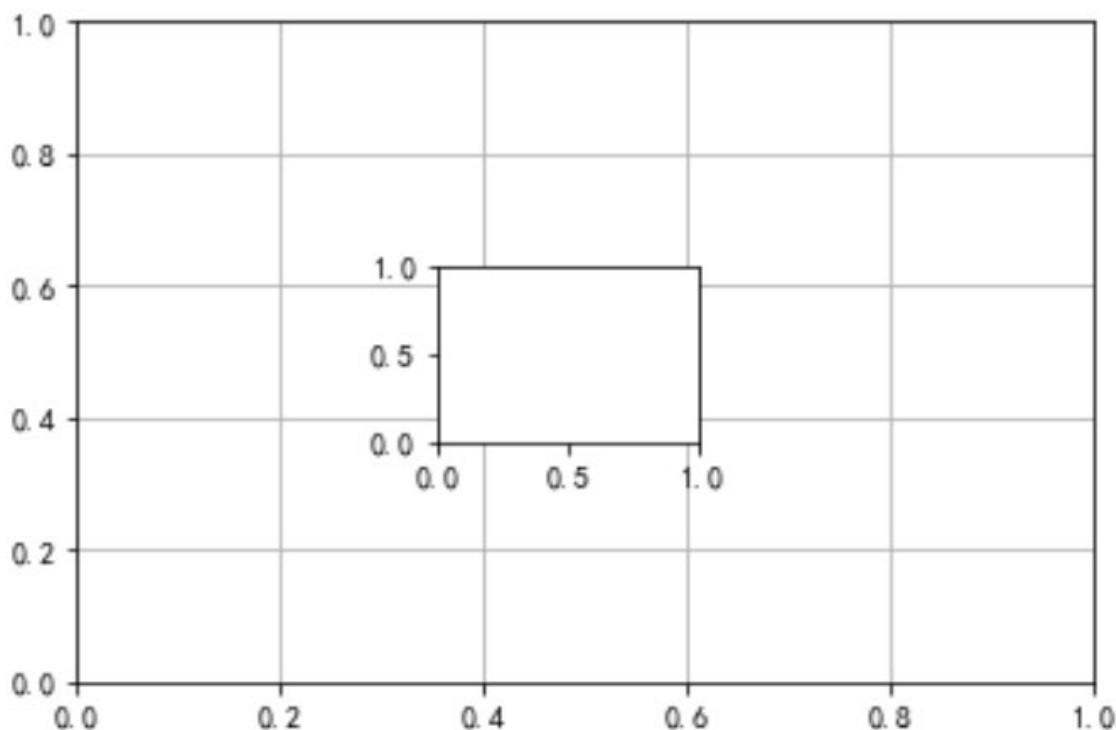
- 很多时候，需要从不同的角度对比和分析数据。这可以通过将多个图放置在一起进行对比。Matplotlib中通过子图subplot的概念实现这一功能。
- 通过plt.axes函数可以创建基本子图，默认情况下它会创建一个标准的坐标轴，并填满整张图。但是可以通过参数指定子图的位置和大小。这个函数的参数是个列表形式，有四个值，从前往后，分别是子图左下角基点的x和y坐标以及子图的宽度和高度，数值的取值范围是0-1之间，画布左下角是（0，0），画布右上角是（1，1）。

● 使用Matplotlib绘图



➤ 子图

```
import matplotlib.pyplot as plt
#使用默认配置，布满整个画布并画上坐标轴
ax1 = plt.axes()
plt.grid()
#在画布中间绘制一个子图
ax2 = plt.axes([0.4, 0.4, 0.2, 0.2])
plt.show()
```



● 使用Matplotlib绘图



➤ 子图

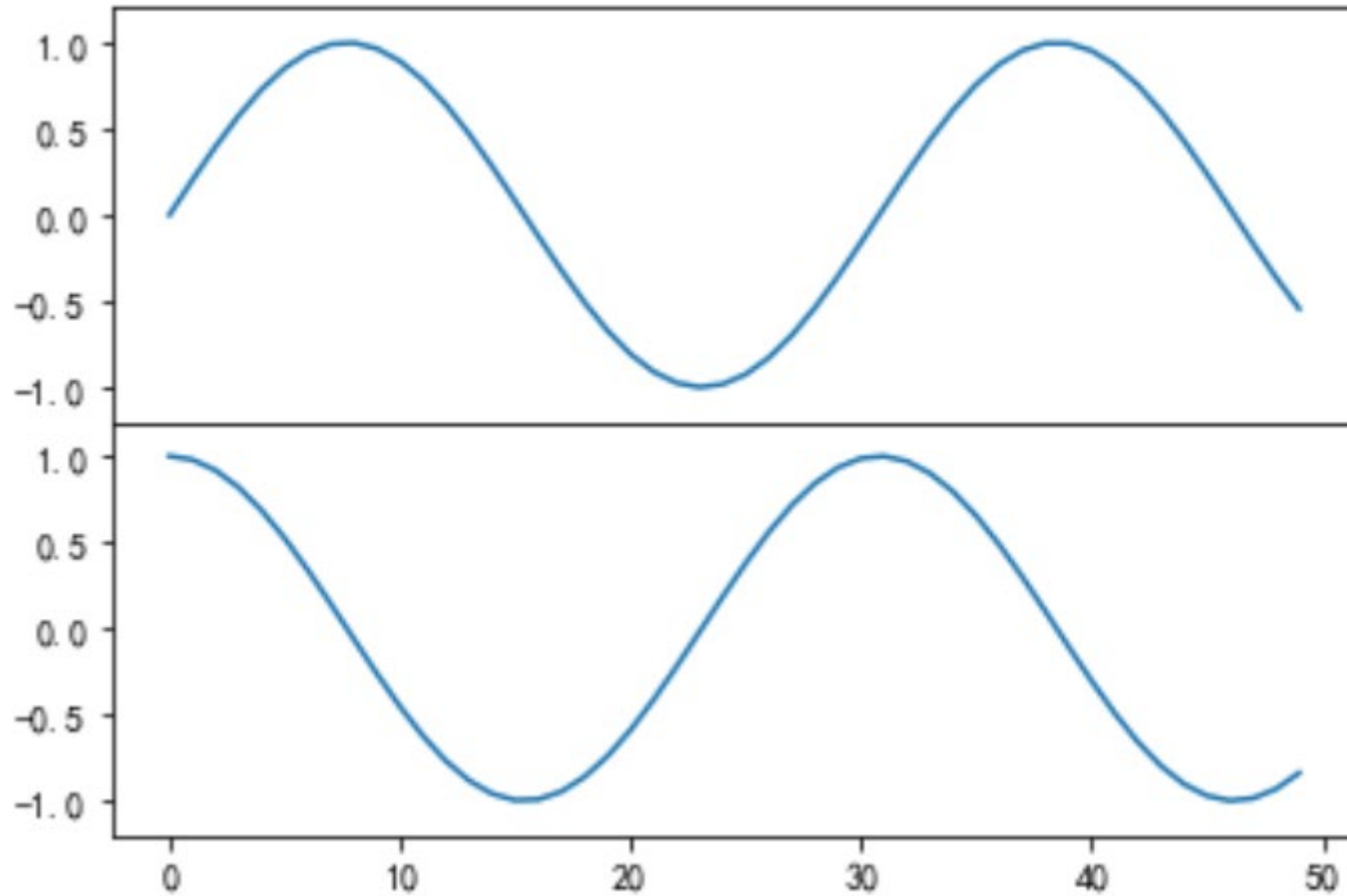
- 上面是Matlab接口的风格，面向对象画图接口中有类似的fig.add_axes()方法可以增加新的子图。下面的例子在两个子图里显示了sin和cos的变化趋势。

```
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure()
ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4], xticklabels=[],
                    ylim=(-1.2, 1.2))
ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4], ylim=(-1.2, 1.2))
x = np.linspace(0, 10)
ax1.plot(np.sin(x))
ax2.plot(np.cos(x))
plt.show()
```

● 使用Matplotlib绘图



➤ 子图



● 使用Matplotlib绘图



➤ 子图

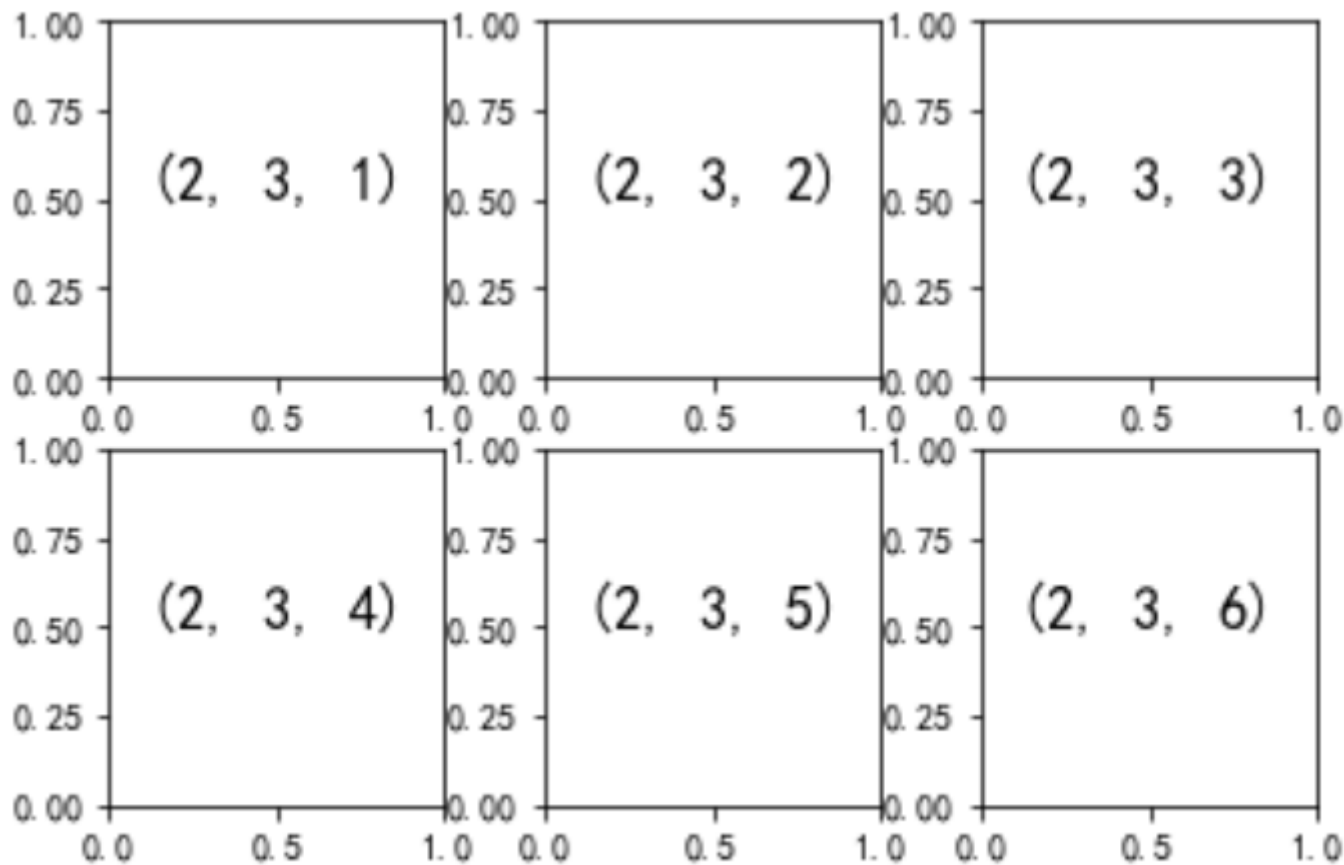
- 第二种方式为使用plt.subplot方法，该方法有三个整数参数，分别表示行数、列数和子图索引值。索引值从1开始，从左上角到右下角依次自增。plt.subplots_adjust方法可以指定子图的坐标和间距，参数分别为子图上下左右的坐标，以及间距(wspace)和上下的间距(hspace)。

```
import matplotlib.pyplot as plt
for i in range(1,7):
    plt.subplot(2,3,i)
    #调整子图间距
    plt.subplots_adjust(wspace=0.3, hspace=0.2)
    plt.text(0.5,0.5,str((2,3,i)),fontsize=16,ha='center')
plt.show()
```

● 使用Matplotlib绘图



➤ 子图



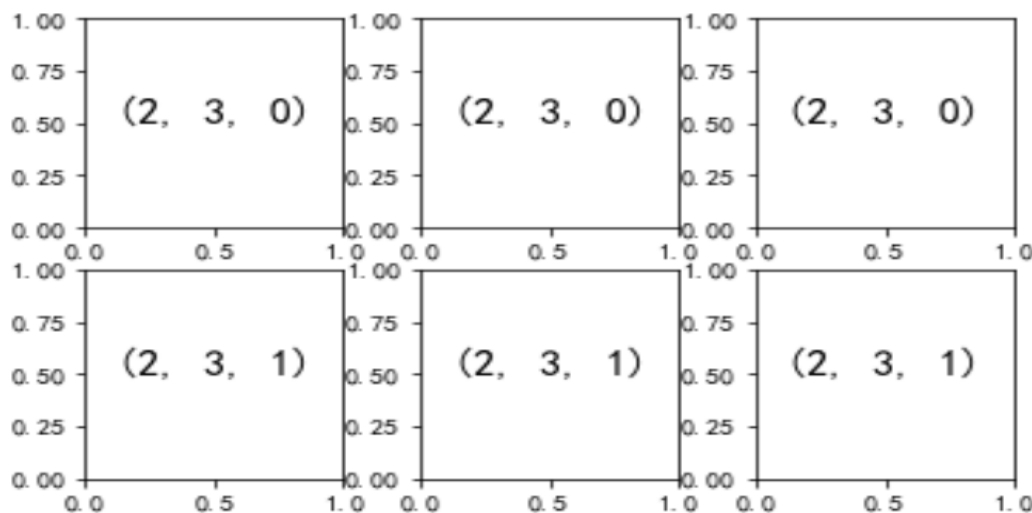
● 使用Matplotlib绘图



➤ 子图

- 使用面向对象的方法`fig.add_subplot()`也可以实现同样的效果。

```
import matplotlib.pyplot as plt
fig = plt.figure()
fig.subplots_adjust(hspace=0.2, wspace=0.3)
for i in range(1, 7):
    ax = fig.add_subplot(2, 3, i)
    ax.text(0.5, 0.5, str((2, 3, i)), fontsize=16, ha='center')
plt.show()
```



➤ 子图

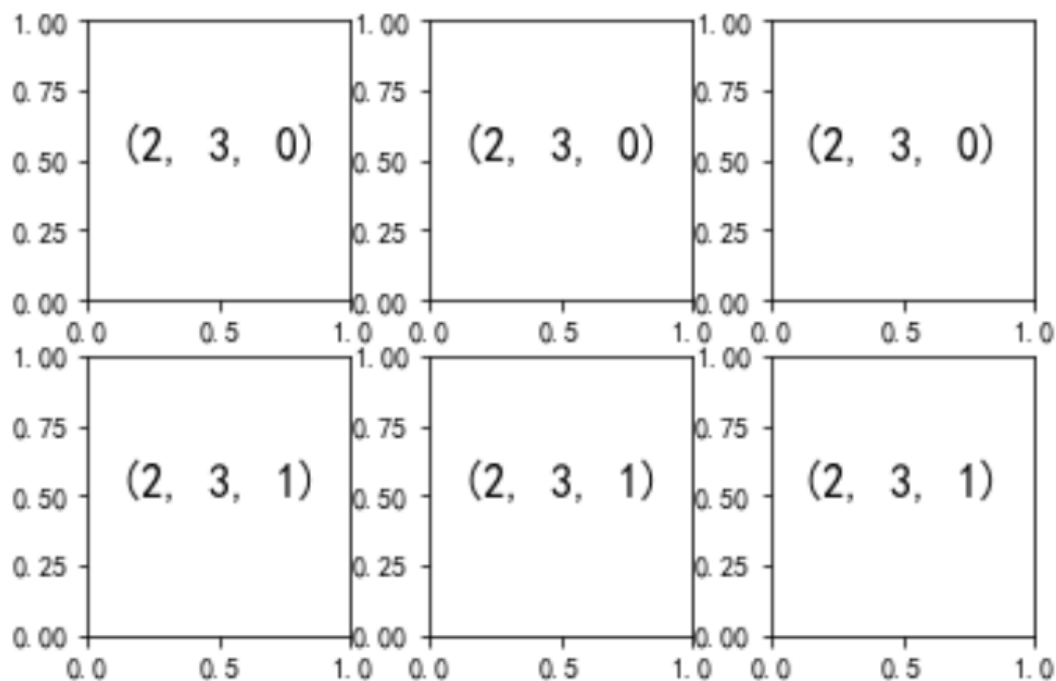
- 使用`plt.subplots()`方法可以快速地创建多子图环境，并返回一个包含子图的Numpy数组。通过对返回的`ax`数组进行调用，可以操作每个子图，绘制图形。但是需要注意的是，`subplot()`和`subplots()`两个方法在方法名上差个字母s外，`subplots`的索引是从0开始的。另外`subplots()`方法还可以通过两个布尔型参数`sharex`和`sharey`，指定是否只显示最外侧的x或y轴的标签。

● 使用Matplotlib绘图



➤ 子图

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 3)
fig.subplots_adjust(hspace=0.2, wspace=0.3)
for i in range(2):
    for j in range(3):
        axes[i, j].text(0.5, 0.5, str((2, 3, i)),
                        fontsize=16, ha='center')
plt.show()
```



● 使用Matplotlib绘图



➤ 子图

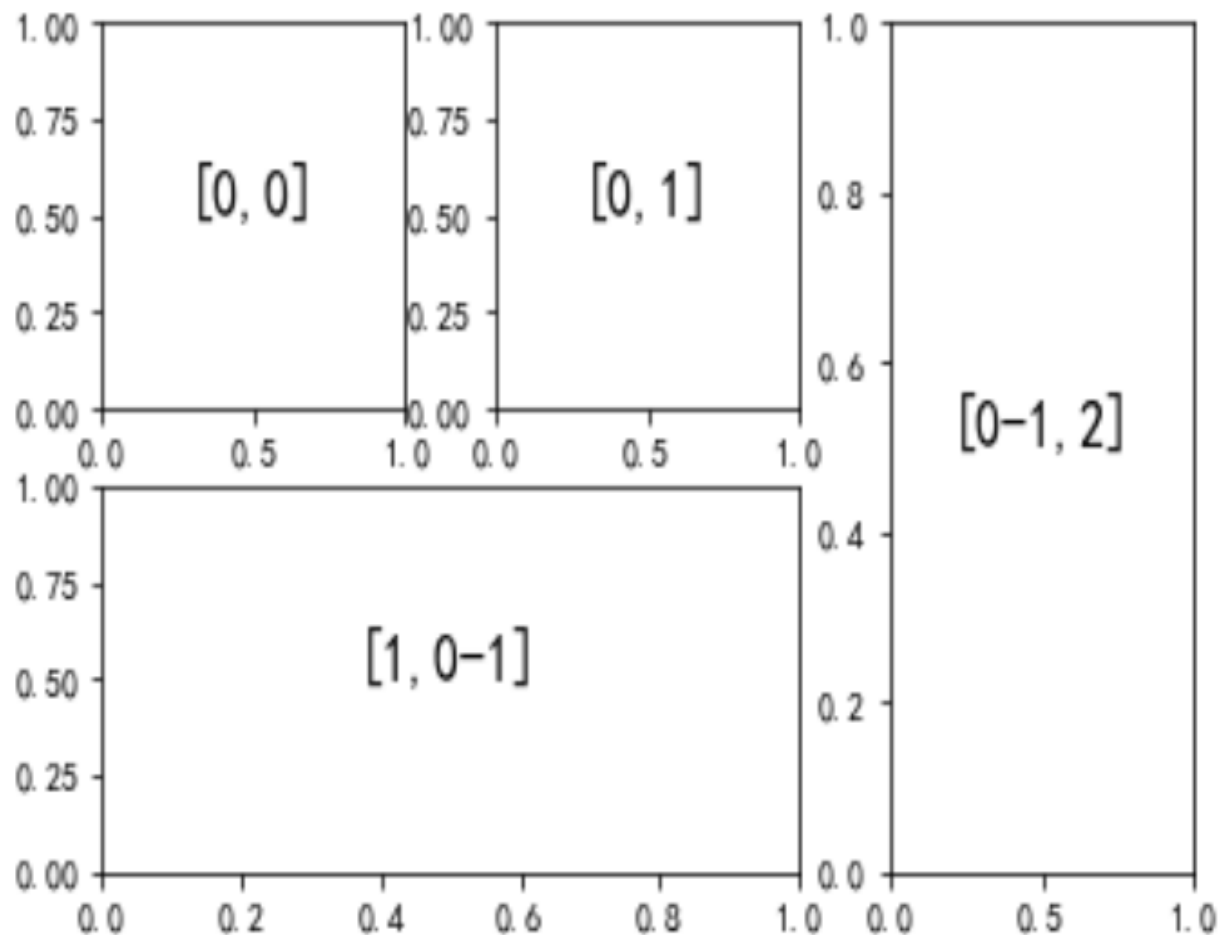
- 对于不规则的子图，可以使用复杂网格的方式进行组织。复杂网格首先指定一个多行多列的网格，然后每个子图占用相邻的一个或多个网格绘制。

```
import matplotlib.pyplot as plt
#生成两行三列的网格
grid = plt.GridSpec(2, 3, wspace=0.3, hspace=0.2)
plt.subplot(grid[0,0]) #将0,0的位置使用
plt.text(0.5, 0.5, "[0,0]", fontsize=16, ha='center')
plt.subplot(grid[0,1]) #将0,1的位置使用
plt.text(0.5, 0.5, "[0,1]", fontsize=16, ha='center')
plt.subplot(grid[1,:2]) #使用1,1-2的位置
plt.text(0.5, 0.5, "[1,0-1]", fontsize=16, ha='center')
plt.subplot(grid[:2,2]) #使用1-2,2的位置
plt.text(0.5, 0.5, "[0-1,2]", fontsize=16, ha='center')
plt.show()
```

● 使用Matplotlib绘图



➤ 子图



➤ 中文显示

● matplotlib绘制图像的时候如指定了中文的坐标轴标签或标题，显示的时候中文字符会显示为小方格子。造成这个现象的原因是matplotlib库的配置信息里面没有中文字体的相关信息。通过修改matplotlibrc配置文件可设置字体相关参数。但是更常用的方法是在程序中动态设置字体参数。下面的两种方式都可以达到同样的目的。

● 通过pyplot进行设置：

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei'] #指定默认字体正确显示中文
plt.rcParams['axes.unicode_minus']=False #用来正常显示负号
```


● 使用Matplotlib绘图



➤ 中文显示

● 通过pylab进行设置:

```
from pylab import mpl
mpl.rcParams['font.sans-serif']=['SimHei'] #指定默认字体正确显示中文
mpl.rcParams['axes.unicode_minus']=False #用来正常显示负号
```

● 具体支持哪些文字，可以通过font_manager.py文件进行查看。FontManager类在初始化时，会遍历

plt.rcParams['datapath']目录下的fonts子目录，并将其中的字体加入可用字体列表。下面列出了一些常用的指定字体：

- 黑体 SimHei
- 仿宋 FangSong
- 楷体 KaiTi
- 微软雅黑 Microsoft YaHei
- 微软正黑体 Microsoft JhengHei

● 使用Matplotlib绘图

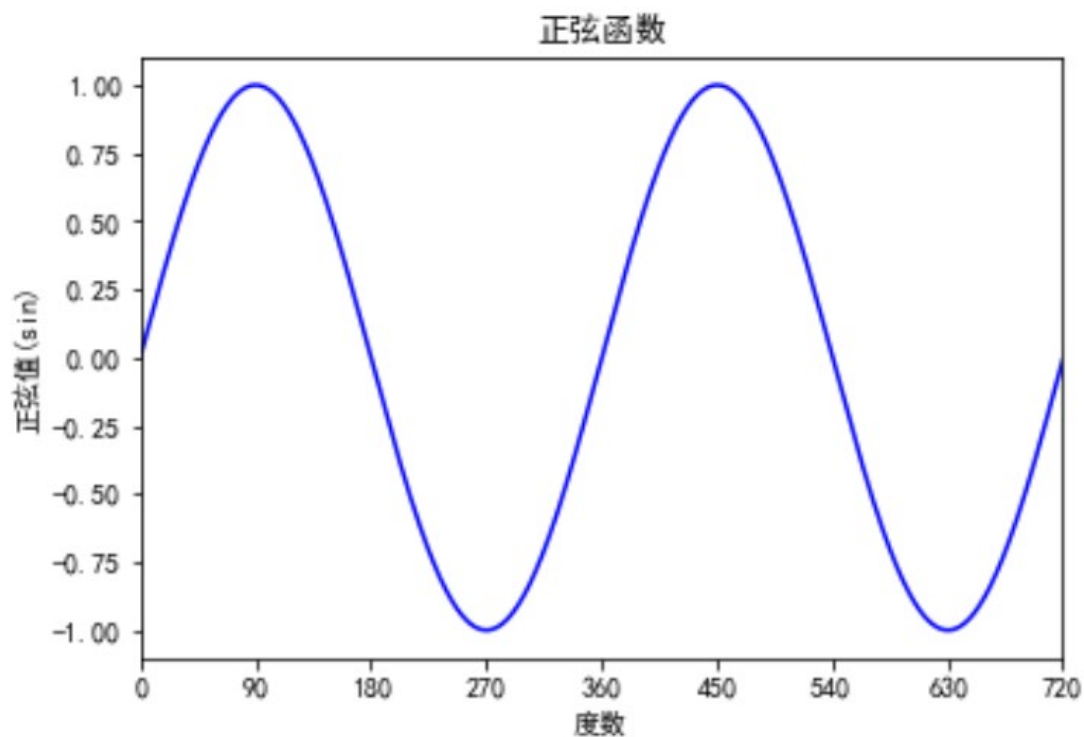


➤ 中文显示

- 下面的程序修改了中文的坐标轴标签和标题。

```
import matplotlib.pyplot as plt
import math
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
#用来正常显示负号
plt.rcParams['axes.unicode_minus']=False
x=range(721)
y=[math.sin(i*3.14/180) for i in x]
plt.axis([0, 720, -1.1, 1.1])
plt.plot(x, y, 'b')
plt.xticks(range(0, 721, 90))
plt.xlabel("度数")
plt.ylabel("正弦值(sin)")
plt.title("正弦函数")
plt.show()
```

➤ 中文显示

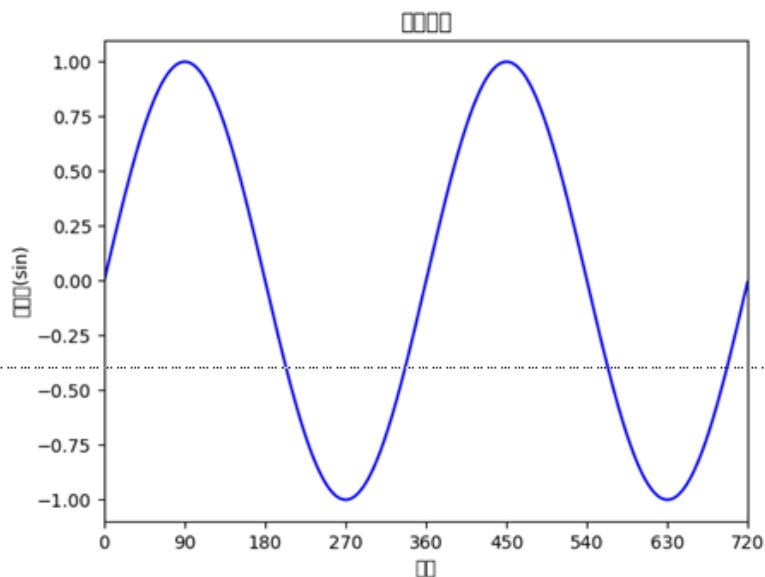


● 使用Matplotlib绘图

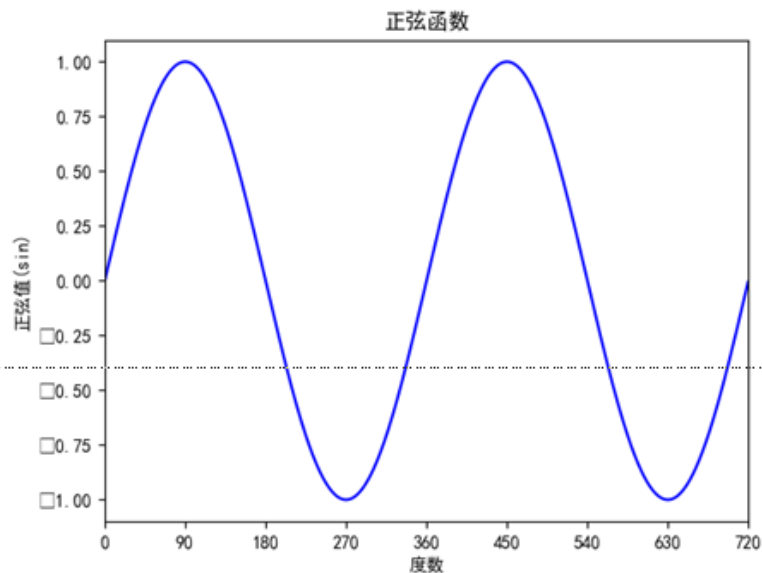


➤ 中文显示

- 其中未指定中文字体和坐标轴使用unicode时的结果为(a)，指定中文字体，但是未指定坐标轴使用unicode时的结果为(b)。



(a)中文字符显示不正常



(b)中文字符正常，负号不正常

● 使用Matplotlib绘图



➤ 中文显示

- 除了使用matplotlib已支持的字体外，也可以使用字体管理器font_manager从文件中读取系统的字体文件。

```
from matplotlib.font_manager import FontProperties  
font = FontProperties(fname=r"C:\\WINDOWS\\Fonts\\STXINGKA.TTF",  
size=14)  
plt.title("正弦函数",fontproperties=font)
```

● 使用Matplotlib绘图



➤ 中文显示

- 利用这种方法，可以在一幅图中同时使用多种字体。

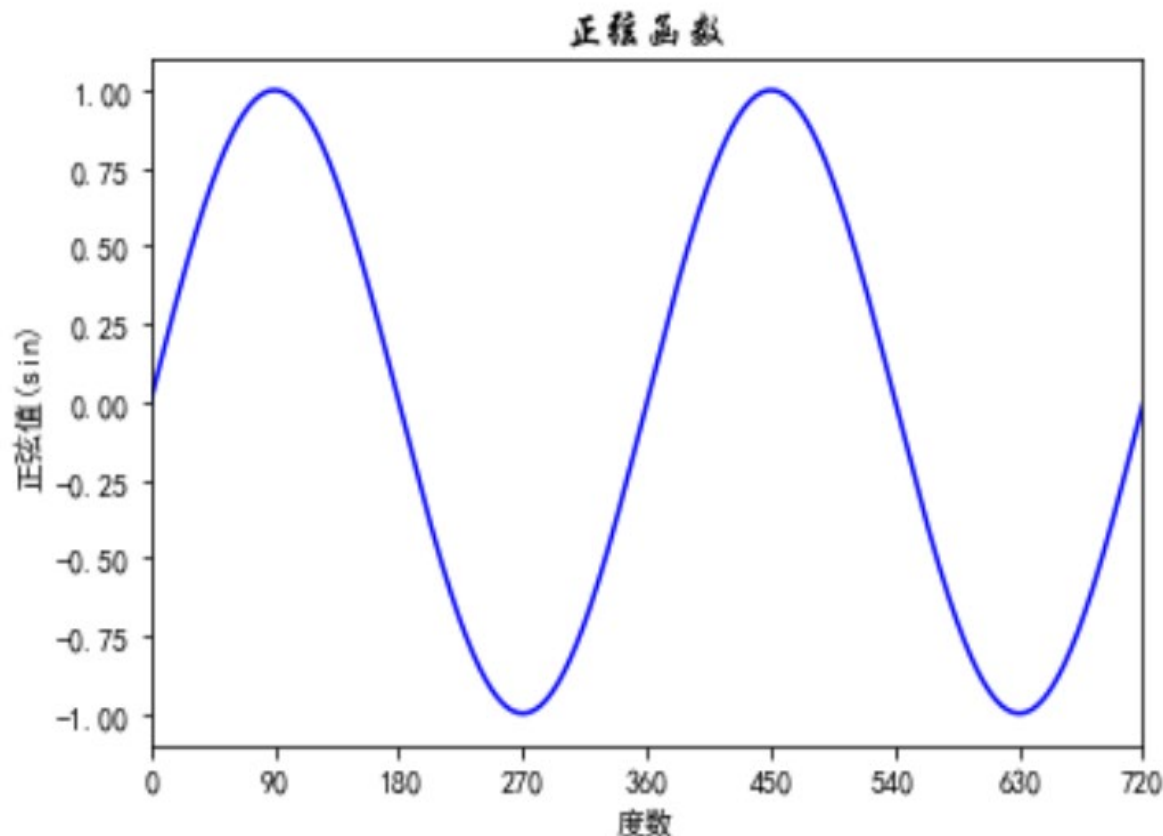
```
from matplotlib.font_manager import FontProperties
font = FontProperties(fname=r"C:\\WINDOWS\\Fonts\\STXINGKA.TTF",
                     size=14)
plt.title("正弦函数", fontproperties=font)
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
#用来正常显示负号
plt.rcParams['axes.unicode_minus']=False
x=range(721)
y=[math.sin(i*3.14/180) for i in x]
plt.axis([0, 720, -1.1, 1.1])
plt.plot(x, y, 'b')
plt.xticks(range(0, 721, 90))
plt.xlabel("度数")
plt.ylabel("正弦值(sin)")
plt.show()
```

● 使用Matplotlib绘图



➤ 中文显示

- 运行结果如图所示，标题使用了从字体文件读取的字体-华文行楷。两个坐标轴的标签仍使用了黑体。



➤ 组合图形与标注

- 折线图，条形图都可以同时展示多个数据序列。但是当图表中不同数据序列的数字变化很大，或表示不同含义时（如成交量和变化率），可以选择增加一个次要坐标轴。如使用条形图结合主要坐标轴展示几种商品在不同月份的成交量。用折线图结合次坐标轴展示变化率。次坐标轴在显示柱形图和折线图组合的图表中表现非常好。

- Matplotlib中可以在图的右侧创建第二个y轴作为次坐标轴，也可以在图的上边创建第二个x轴作为次坐标轴。一般的用法是建立第二个y轴。首先调用`subplots()`函数创建子图，在获取的第一个坐标轴Axes上画主坐标轴相关的数据，然后调用`twinx()`函数生成镜像的y轴次坐标轴，在获取的第二个坐标轴上绘制次坐标轴相关的数据。

● 使用Matplotlib绘图

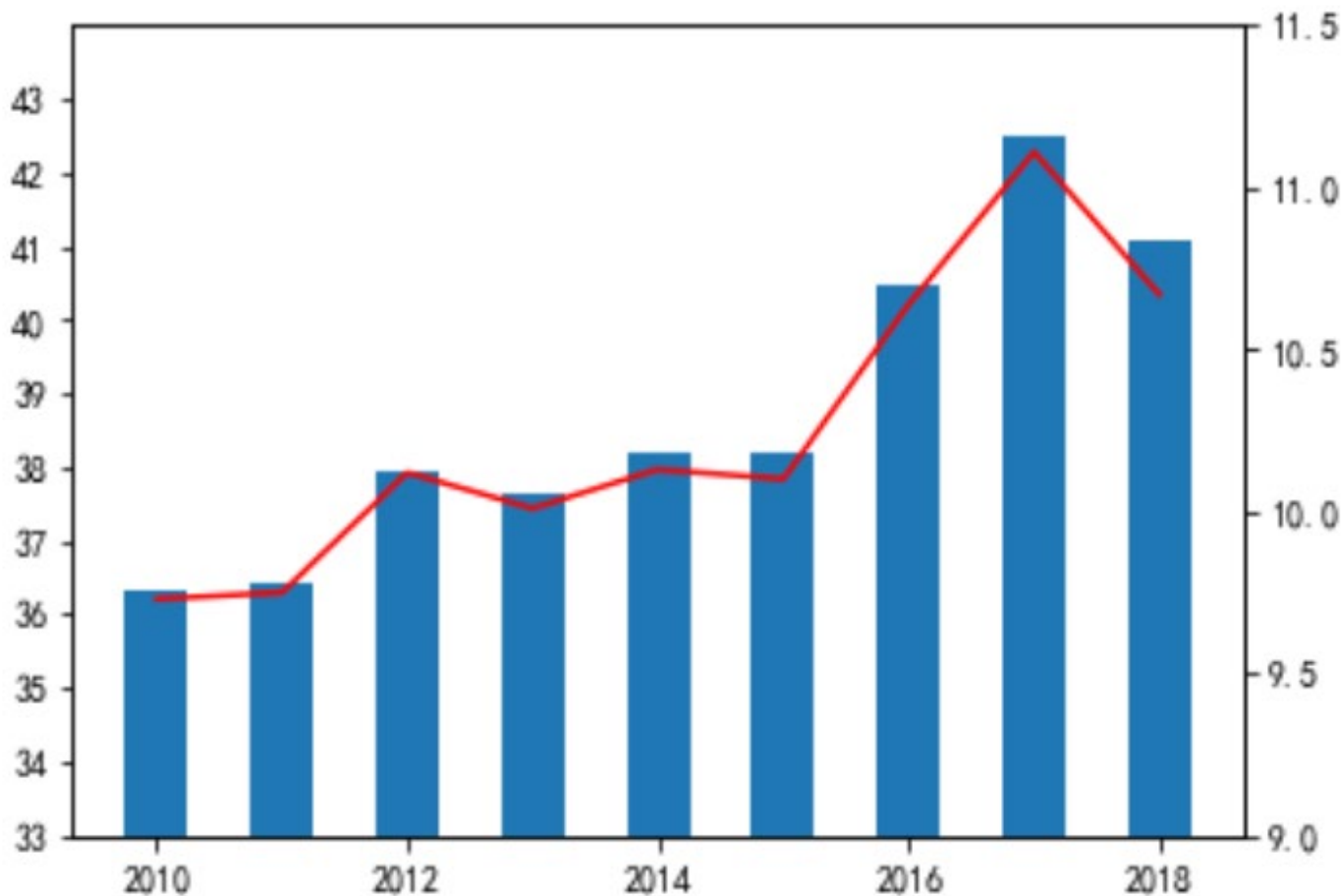


➤ 组合图形与标注

- 下面的例子使用主坐标轴绘制了陕西省2010年至2018年出生率和出生人口，数据来源为陕西省统计局 (<http://tjj.shaanxi.gov.cn/126/111/19566.html>)。

```
import matplotlib.pyplot as plt
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
#数据来自陕西省统计局
#http://tjj.shaanxi.gov.cn/126/111/19566.html
population=[36.34, 36.45, 37.93, 37.62, 38.18,
            38.22, 40.46, 42.48, 41.08]
year=range(2010, 2019)
growth=[9.73, 9.75, 10.12, 10.01, 10.13, 10.1,
        10.64, 11.11, 10.67]
fig, ax1=plt.subplots()
ax1.bar(year, population, width=0.5, label=
        "出生人口(万人)")
ax1.set_ylim([33.00, 44.00])
ax1.set_yticks(range(33, 44))
ax2=ax1.twinx() #绘制y轴的次坐标轴
ax2.plot(year, growth, "r", label="出生率(%)" )
ax2.set_ylim([9.00, 11.5])
plt.show()
```

➤ 组合图形与标注



➤ 组合图形与标注

- 图可以表达丰富的信息。除了展示基本的数据外，还可以通过辅助线，数据标签，图例突出要表现的主题。上图仅画出了基本的条形图和折线图，只能看到数据变化的趋势，看不到具体的数值。通过网格或增加横线(或竖线)可以得到相对准确的估计。
- Matplotlib中辅助线相关的函数包括：
- `---grid()` : 显示网格线
- `--hlines(y, minx, maxx)` : 在指定的y处从minx到maxx画横线。
- `--vlines(x, miny, maxy)` : 在指定的x处从miny到maxy画竖线。

● 使用Matplotlib绘图



➤ 组合图形与标注

- 下面的例子在上个例子的基础上增加了标题（title函数），横线（hlines函数），数据标注（text函数），注解（annotate函数），并修改了x轴的显示方式(xticks)。

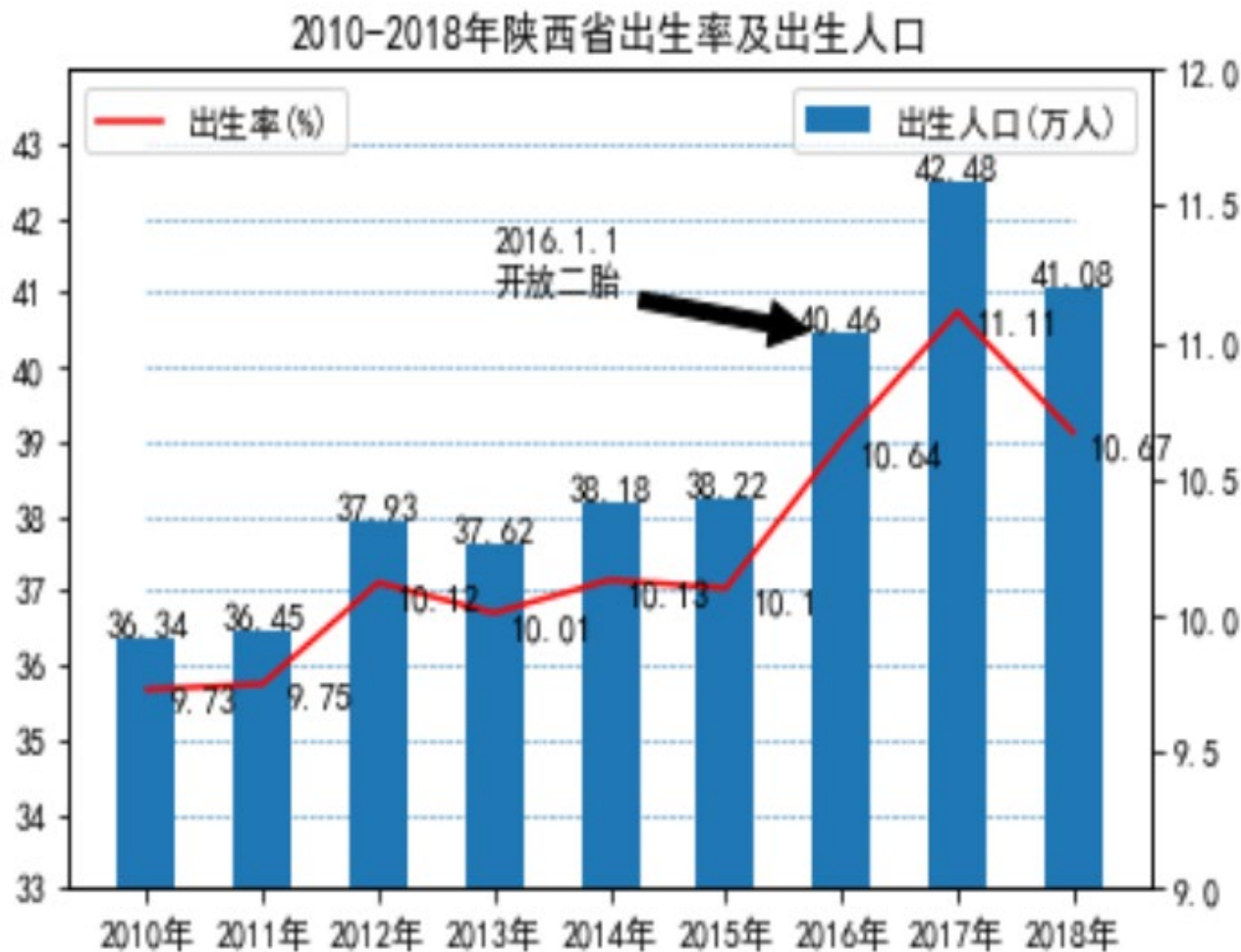
➤ 组合图形与标注

```
import matplotlib.pyplot as plt
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
#数据来自陕西省统计局
#http://tjj.shaanxi.gov.cn/126/111/19566.html
population=[36.34, 36.45, 37.93, 37.62, 38.18, 38.22,
            40.46, 42.48, 41.08]
year=range(2010, 2019)
growth=[9.73, 9.75, 10.12, 10.01, 10.13, 10.1, 10.64,
        11.11, 10.67]
marker=[str(x)+'年' for x in year]
fig, ax1=plt.subplots()
ax1.bar(year, population, width=0.5, label="出生人口(万人)")
for x, y in zip(year, population):
    plt.text(x, y, str(y), ha='center', va='bottom', fontsize=10.5)
ax1.set_ylim([33.00, 44.00])
ax1.set_yticks(range(33, 44))
ax1.hlines(range(34, 44), 2010, 2018, linestyle="--", linewidth=0.5)
ax1.annotate("2016.1.1\n开放二胎", xy=(2015.8, 40.46), xytext=(2013, 41),
            arrowprops=dict(facecolor='black', shrink=0.05))
ax1.legend(loc=1)
ax2=ax1.twinx()
ax2.plot(year, growth, "r", label="出生率(%)")
for x, y in zip(year, growth):
    plt.text(x+0.5, y-0.1, str(y), ha='center', va='bottom', fontsize=10.5)
ax2.set_ylim([9.00, 12])
ax2.legend(loc=2)
plt.xticks(year, marker)
plt.title("2010-2018年陕西省出生率及出生人口")
plt.show()
```

● 使用Matplotlib绘图



➤ 组合图形与标注



● 使用Matplotlib绘图



➤ 保存图片

函数名称	函数作用
plt.savefig	保存绘制的图片，可以指定图片的分辨率、边缘的颜色等参数。

```
plt.savefig(save_path, format='png', transparent=True, dpi=300,  
pad_inches = 0)
```

➤ 保存图片

```
plt.savefig(save_path, format='png', transparent=True, dpi=300,  
pad_inches = 0)
```

参数	描述
fname	包含文件路径或Python文件型对象的字符串。图片格式是从文件扩展名中推断出来的（例如pdf格式的.pdf）
dpi	设置每英寸点数的分辨率，默认为100
facecolor , edgecolor	子图之外的图形背景颜色，默认是'w'（白色）
format	文件格式（'png','pdf','svg','ps'等）
bbox_inches	要保存的图片范围，如果设置为 'tight '则去除图片周围的空白

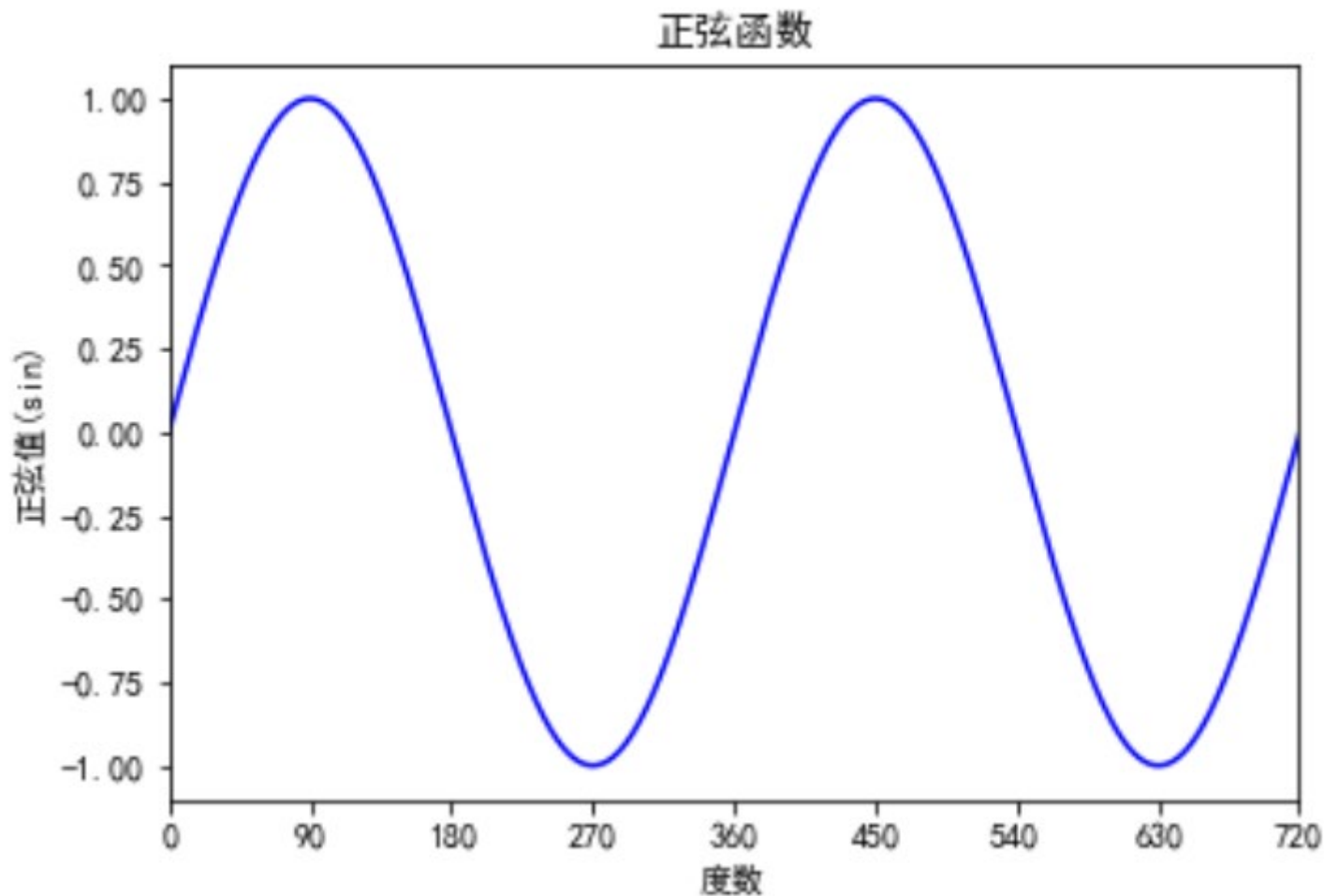
➤ 保存图片

```
import matplotlib.pyplot as plt
import math
#用来正常显示中文标签
plt.rcParams['font.sans-serif']=['SimHei']
#用来正常显示负号
plt.rcParams['axes.unicode_minus']=False
x=range(721)
y=[math.sin(i*3.14/180) for i in x]
plt.axis([0, 720, -1.1, 1.1])
plt.plot(x, y, 'b')
plt.xticks(range(0, 721, 90))
plt.xlabel("度数")
plt.ylabel("正弦值(sin)")
plt.title("正弦函数")
plt.savefig('C:/Intel/sin.png', dpi=300)
```

● 使用Matplotlib绘图



➤ 保存图片



● 使用Matplotlib绘图

- 数据可视化是展示数据分析结果的最终环节。使用数据画出图表容易，但是通过图表清晰表达观点并非易事，它绝非一朝一日之功，但也并不是无规律可循。
- 首先要充分理解数据分析的目的，比如商业目的或要支持什么结论。
- 其次是灵活选择图表，并调节图表坐标系以合适的形式进行可视化展示。

● 使用Matplotlib绘图

- 最后从图表数据的分析中发现问题，确认是否表达了数据要表达的观点，是否重点突出，一目了然。
- 数据可视化需要经过多次重复迭代的过程，不断优化图表设计元素。在设计的过程中，也不要追求图表效果而将图表画的过于复杂，从而使他人难以看出意图。

- 在图表的设计中，也有一些小技巧。
- 在图表中要突出主次和重点。图表的实线、虚线的选择都要围绕主题，重要的用实线或深颜色，次要的用虚线或浅颜色。
- 数据序列的排列的顺序也要有一定的理由。比如多种方案的性能对比，可以按方案提出的时间排序，也可以按方案的设计思想分类排序。
- 使用数据标签可以清楚展示具体数据，是否需要数据标签取决于数据的数量，以及读者是否关心具体数据。有时候文字注释和说明可以突出图表的目的和主题。

- 在一个图表中不要使用太多的颜色。论文中的图表也要考虑使用黑白打印机时的效果，能通过灰度、线型和标注形状区分数据序列的最好不要过于依赖颜色。
- 颜色的选择要保持一致，比如在图表1中红色表示方案1，在其他的图中也用红色表示同一方案。一篇文章中所有的图表风格要保持统一，包括配色，字体字号，箭头等。